Wireless Networks (2024) 30:711–735
https://doi.org/10.1007/s11276-023-03515-1

ORIGINAL PAPER

# Coalescing novel QoS routing with fault tolerance for improving QoS parameters in wireless Ad-Hoc network using craft protocol

R. Aruna[1] · Virendra Singh Kushwah[2] · S. Phani Praveen[3] · Rahul Pradhan[4] · Amit Jaykumar Chinchawade[5] · Renas Rajab Asaad[6] · R. Lakshmana Kumar[7]

Accepted: 7 September 2023 / Published online: 4 October 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 202

**Abstract**
The information sharing on behalf of the delivery of the packet in the network router and the original information that is share to the correct person is enabled for the analysis and then the analysis is done. The traffic related problem based on the sharing the information is occur when two or more sis sent in the same routing path. So the packet or the information has been sent using the single router or an execution time delivery for the analysis. The method of novel QoS Routing, fault tolerance is used in this study for the analysis of the system to produce the formation and the analysis of the packet routing. Fault tolerance is used in the deliver the packets without fail and to deliver it in the fast manner. The QoS is used for managing the information or the packets which makes and sends the data which is the high priority of the data and the analysis of the dependency is analysed. The finding of the CRAFT related protocol and the efficient routing has been found. Routing system the effective method for the analysis and the protocol transfer has been enabled using the CRAFT method. The suggested system explains the algorithms for enhancing fault detection, fault isolation, data redundancy, robustness against coalescing innovative QoS routing, and fault tolerance improving QoS parameters in the ad-hoc wireless network utilising CRAFT protocol. Compared to the earlier existing system, fault detection is now three times better. The execution time is 96% faster, and the decreased latency is less than 2 ms. The data redundancy rate is likely about 90% when restricted to the current services, which have improved in terms of resilience by 93%, jitter by 7%, and packet arrival time by over 50 ms per transmission.

**Keywords** CRAFT protocol · Data redundancy · Execution time · Fault detection · Fault tolerance · Novel fault tolerance · Novel QoS routing protocol

RETRACTED ARTICLE

## 1 Introduction

The information sharing and the routing of the information in the network path has been monitored in the perfect way nowadays. Here the sharing of the information and the delivery of the packet is done using the coalescing novel QoS routing. This is used for the research related issues which helps in the classification of the routing protocol and the probabilistic and the deterministic. Also, here the real time and the routing protocol [1] enable the quality of the service and the analysis between the sensor and the bandwidth requirements to enables it in the energy constraints for the analysis is done. Here the process of marking, classification, policing, avoidance and queuing has been enabled for the analysis in this paper. Then the process of fault tolerance means the ability to manage the continuous interruption of the components and the ability to manage the despite and the operating the computer system and the networking that enables the fault tolerance is proposed. Here the process of maintaining the hardware and the software errors and the induction of the changes in the physical analysis is done [2]. Here some of the main tolerance that has been enabled for the reverse and the strike-slip has been enabled for the analysis is done. This makes the backups and the storing of the data to maintain the operations and the mirroring the identical operating system to produce the data and the information sharing. Here for the proposed method the CRAFT method is used for the analysis of the system to produce the Limitations and the bottlenecks to make the throughput for helping the

Extended author information available on the last page of the article

information to share. Here the packet arrival and the redundancy of the data that enables the fault detection and the delay in the robustness is calculated. Here the improvement of the routing factors and the data transmission for each protocol enables the efficient routing method for the fast transfer of the data in the networks [3]. For the detection of the fault in the packet, delay and the robustness enables the routing system and the efficient routing can be enabled [4]. Here regarding the results, the robustness, data reduction analysis, fault analysis, time of execution, delay, and the jitter comparison has been analyzed based on the information sharing. This makes the execution of the system to produce the packet delivery very soon and this makes the formation of the system to produce the time against the routing analysis and this enables the collection of the novel analysis of the information sharing. The routing factors are also enables the very shortest path and the analysis of the shortest path can be enabled for the analysis in those study. The main contribution of this study is as follows:

- The routing factor identification and the information sharing is done using the CRAFT.
- The finding of the packet delivery and the execution time of the packet delivery is analyzed in this study using the novel QoS Routing.
- The improvement of the CRAFT protocol can be enabled in this study.

The sectional divisions are as follows: Sect. 1 shows the overall process of the paper in the introduction format, in Sect. 2 the existing paper has been reviewed as a Literature review. In Sect. 3 the methods have been elaborated by involving the proposed system. In Sect. 4 the results have been executed and in Sect. 5 the conclusion regarding the paper has been written with the future work.

## 2 Literature review

Song et al. [5] discussed about the scheduling and the routing related transmission has been enabled for the industrial internet services and the transmissions. In this paper the method of OPC UA for the PubSub for the scheduling and the routing purpose is done. Then based on the simultaneous transmission the data streaming is done and for the scheduling of the packets the scheduling algorithm has been used. For analyzing the speed and the reliability the time triggered traffic based colony algorithm has been enabled for the shortest path in the scheduling calculations. Zhuo et al. [6] examine about novel the security threats in the survey in the routing and the detection of the information in the space networks has been enabled for the analysis. Also, here the anomaly detection of the router path [7] is done. Then the deep learning method is used for the analysis and the information network security enables the future exploration can be enabled. Guo et al. [8] talked over about study, the software defined network and the analysis using the Scheduling algorithm has been used for the problems in the multi path traffic scheduling has been enabled for the linking the deep reinforcement learning the weight calculation algorithm has been enabled for the deep reinforcement learning has been enabled and proposed in this study. Monika et al. [9] considers about analysis of the random use the GMPLS network has been proposed and for the analysis of the decision tree work the Data protection algorithm work has been enabled for the optical networks. Then the probability of the blocking the latency and the parameters has been proposed. This results the QA that reduces the latency and the enhancement for the data protection has been enabled. Wang et al. [10] has a discussion about the security related segmentation and the SDP related enhancement has been enabled [11] for the detection of the label has been enabled in this study. The security related segmentation and the analysis of the packet delivery is proposed in this study. Here the data plane and the security cost that is enabled for the information is analyzed as well. Marwah et al. [12] discussed about the ensemble based machine learning technique which is done using the VANET mobility and for the routing based machine learning technique can be used. The machine learning technique is used for the decision making and the comparative analysis has been enabled in the performance is enabled. Here the increase in the consumption has been done for the analysis. Song et al. [13] writing about low latency and the reliability of the IoT has been enabled for the end-to-end challenges in the internet has been enabled. The IoT enables the transformation and the industries that enables the internet of things.

## 3 Proposed coalescing novel QoS routing and fault tolerance

The QoS Routing and fault tolerance focuses on Coalescing novel QoS Routing and fault tolerance (CRAFT) protocol. It is a new method for improving QoS Routing and fault tolerance protocol against efficient routing. Implementing a new method for efficient routing and enhancing QoS routing and fault tolerance protocols presents several challenges. These include designing a protocol that meets specific network requirements while remaining compatible with existing protocols, ensuring scalability to handle network growth, achieving compatibility and interoperability with current infrastructure, optimizing resource utilization within constraints, addressing security concerns,

RETRACTED ARTICLE

conducting thorough testing and validation, promoting deployment and adoption, and maintaining and upgrading the method over time. Routing is the process of determining the best path for data to travel from its source to its destination across a network. In a computer network, routing is essential for ensuring that data is transmitted efficiently and reliably. During this communication, there is a lot of disturbance occurs [14]. Disturbances like congestion, packet loss, latency, jitter, and network failures can impact data transmission in computer networks. In this proposed system we find the improving method for each factor for efficient routing. This method is called CRAFT (Coalescing novel QoS Routing and fault tolerance protocol). The first protocol is QoS routing, QoS (Quality of Service) is a set of technologies and policies designed to ensure that network resources are used efficiently and effectively, and that network applications receive the appropriate level of service they require. QoS is important because it enables the network to provide predictable performance, which is essential for applications such as real-time voice and video conferencing. Quality of Service (QoS) routing is a type of routing algorithm that takes into account the QoS requirements of network applications when selecting the optimal path for data transmission [15]. QoS and QoS routing are key elements in network design and management, enabling the efficient delivery of different types of traffic with varying QoS requirements. They help to ensure that network resources are utilized effectively and that applications and services receive the necessary performance levels to operate smoothly.

QoS routing is used in networks where different applications have different QoS requirements, such as real-time voice and video applications. Real-time voice and video applications enable instant communication through audio and video. They prioritize low latency, high audio/video quality, and synchronize streams. These applications employ compression, network protocols, and QoS mechanisms for a seamless user experience. Stable connectivity, sufficient bandwidth, and security measures are essential. Figure 1 describes the main objective of QoS routing is to ensure that network traffic is routed over paths that meet the required QoS criteria, such as delay, jitter, packet loss, and bandwidth. QoS routing accomplishes this by taking into account parameters such as packet loss rate, end-to-end delay, and jitter. QoS algorithms can be categorized into two main types: Static and dynamic. In static QoS routing, the QoS requirements of the network applications are predefined, and the routing algorithm selects the path based on the preconfigured QoS parameters. In dynamic QoS routing, the QoS requirements of the network applications may vary over time, and the routing algorithm adjusts the path selection dynamically based on the changing QoS parameters [16]. The factors against effective routing are jitter, packet arrival time, execution time, and delay. In both static and dynamic QoS routing, the QoS requirements of network applications serve as the guiding factors in selecting the optimal path. They ensure that the chosen path aligns with the desired performance levels, guarantees the necessary QoS parameters, and optimizes the overall network performance to support the applications' needs.

In QoS routing, a jitter is used as one of the parameters to determine the optimal path for data transmission. Determining the optimal path for data transmission involves path discovery, metric calculation, path evaluation, and path selection based on criteria such as QoS requirements and network conditions. The goal is to select a path that meets the desired network requirements, minimizes performance issues, and satisfies QoS constraints. Since jitter is used as one of the parameters in QoS routing. Figure 2 describes jitter refers to the variation in the timing of a signal's transitions, which can cause problems in data transmission, digital signal processing, and other applications that rely on precise timing [17]. There are several techniques for improving jitter, including: Clock synchronization: Ensuring that all devices involved in signal transmission are synchronized to the same clock can reduce jitter. This can be done using techniques such as clock recovery, phase-locked loops (PLLs), and delay-locked loops (DLLs). The phase-locked loop (PLL) control system automatically adjusts the phase of a locally generated signal to match the phase of an input signal. PLLs work by producing an oscillator frequency that corresponds to the frequency of an input signal. On the other hand, DLLs are circuits that provide precise control over the relative delay between two signals. They are primarily used for clock and data alignment, skew compensation, and timing adjustments. DLLs are especially useful in synchronous systems where maintaining precise timing relationships is crucial. Noise reduction: Reducing noise in the signal path can help to reduce jitter. This can be done by using shielded cables, minimizing the length of the signal path, and using low-noise components. Signal conditioning: Conditioning the signal to remove any high-frequency components can help to reduce jitter. This can be done using techniques such as low-pass filtering and equalization.

$$\sigma_{\Delta S} = k\sqrt{\Delta S} \tag{1}$$

where, k is proportionality constant, and $\Delta S$ is jitter with measurement interval second.

$$\sigma_{\Delta S} = \zeta \Delta S \tag{2}$$

where, $\zeta$ another constant

RETRACTED ARTICLE

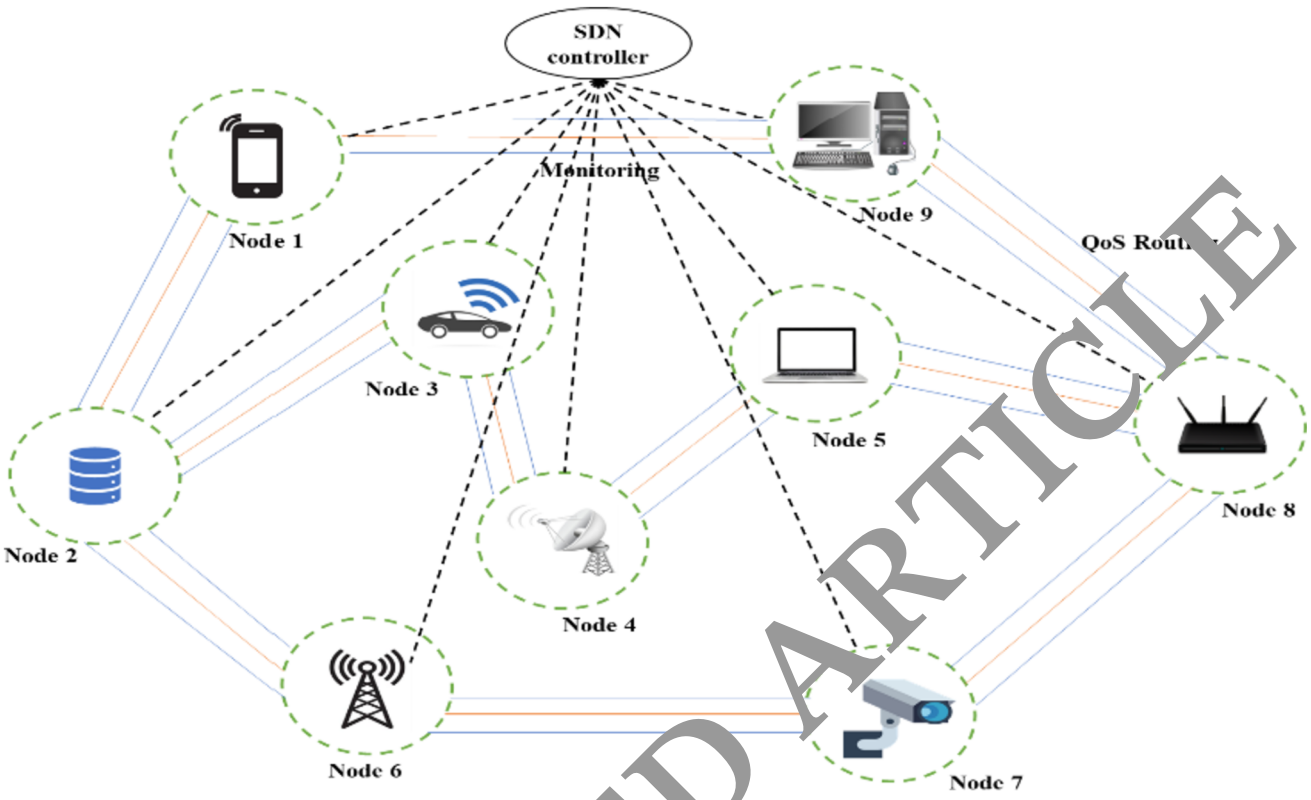



**Fig. 1** QoS routing treating flow

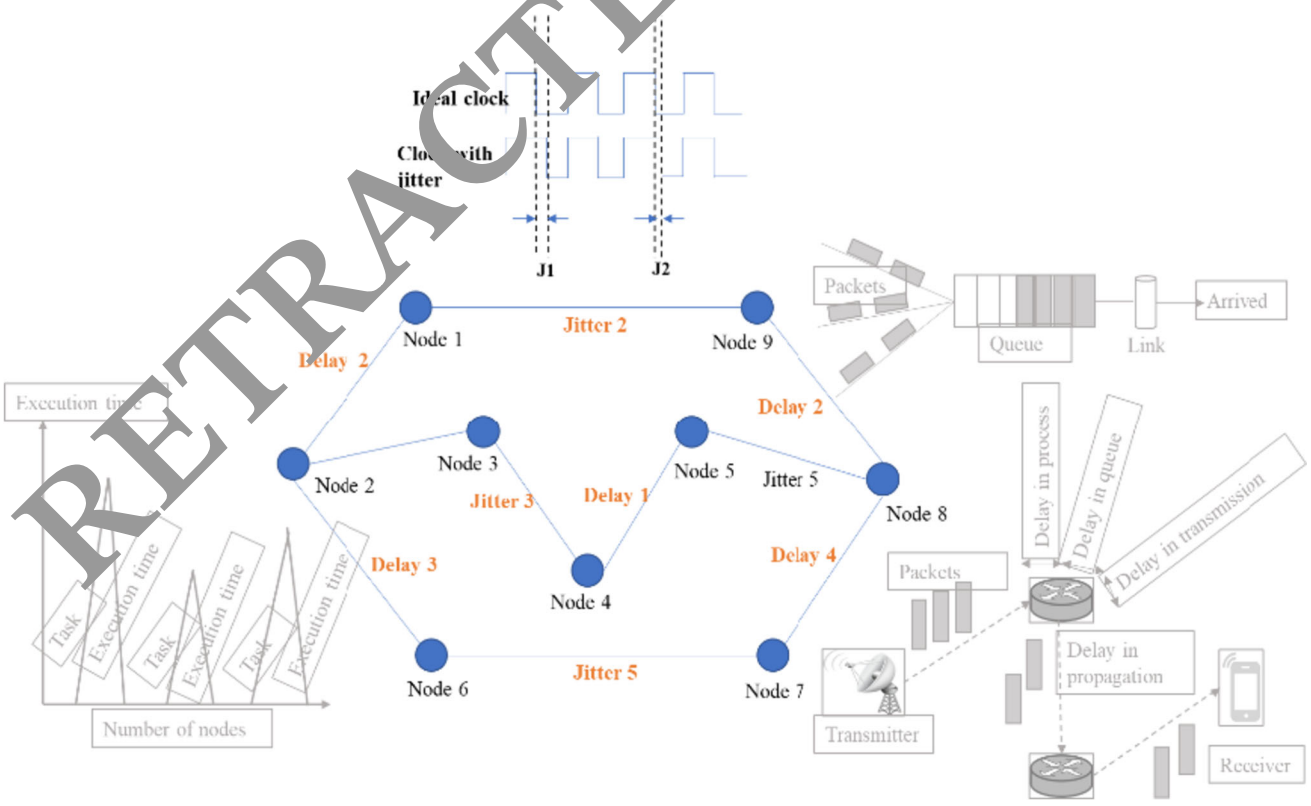



**Fig. 2** Processing Treating for QoS Routing with jitter

RETRACTED ARTICLE

$$\sigma_{\Delta\varnothing} = 2\pi \frac{\sigma_{\Delta S}}{S} = \omega_0 \sigma_{\Delta S} \tag{3}$$

where, S is a period time.

$$\sigma^2{}_{\Delta\varnothing} = \frac{\Gamma^2_{\text{rms}} . \overline{i_n^2}/\Delta f}{2q^2_{\max}} \Delta S. \tag{4}$$

Using Eq. (3) and (4) we can get k constant value.

$$k = \frac{\Gamma_{\text{rms}}}{q_{\max}\omega_0} \sqrt{\frac{1}{2}\frac{i_n^2}{\Delta f}}. \tag{5}$$

Using the above step, we can find the jitter occur time.

Error correction: Implementing error correction techniques such as forward error correction (FEC) can help to reduce the impact of the jitter on the signal. FEC is a technique used to enhance data reliability and reduce the impact of errors or disturbances on a signal by adding redundancy, detecting and correcting errors, and using interleaving to dissipate the effects of jitter. Jitter measurement and analysis: Monitoring and analyzing the jitter in a signal can help to identify the source of the problem and guide the selection of appropriate mitigation techniques. System design: Designing the system to tolerate some amount of jitter can help to reduce the impact of jitter on the overall performance. This can be done by designing the system with appropriate margins, redundancy, and error correction. By using these techniques, it is possible to reduce the impact of jitter on signal quality and improve the overall performance of the system [18]. Jitter can cause a display monitor to flicker, impair the ability of a processor in a desktop or server to perform as intended, introduce clicks or other undesirable effects in audio signals, and result in data loss between network devices. Error correction detects and corrects transmission errors, clock synchronization aligns timing signals, noise reduction minimizes unwanted disturbances, and system design involves planning and implementing a complex system, considering error correction, clock synchronization, and noise reduction.

Jitter refers to the variation in packet delay over time. It occurs when different amounts delay packets due to network congestion or other factors. Factors causing network congestion include high traffic, bottlenecks, insufficient bandwidth, network topology issues, packet loss, QoS mismanagement, and DDoS attacks. Mitigation strategies involve traffic shaping, prioritization, load balancing, QoS implementation, and infrastructure optimization. Jitter is an essential metric for real-time applications, as it can cause noticeable variations in the quality of inputs.

**Algorithm for improving QoS Routing using Jitter factor**

**Step1:** Define a function for calculating the jitter factor

```
function calculate_jitter_factor (previous_delay, current_delay):
    return abs (previous_delay - current_delay)
```

**Step2:** Define a function for calculating the weight for each path based on jitter

```
function calculate_jitter_weight (path, jitter_factor):
    total_jitter = 0
    for i in range (1, len(path)):
        previous_delay = get_delay(path[i-1])
        current_delay = get_delay(path[i])
        current_jitter_factor = calculate_jitter_factor (previous_delay, current_delay)
        total_jitter += current_jitter_factor
    return 1 / (1 + total_jitter * jitter_factor)
```

**Step3:** Define a function for selecting the best path based on jitter weight

```
function select_best_path (paths, jitter_factor):
    best_path = None
    best_weight = 0
    For path in paths:
        weight = calculate_jitter_weight (path, jitter_factor)
        if weight > best_weight:
            best_weight = weight
            best_path = path
    return best_path
```

Initialize network topology and QoS metrics as jitter. Initializing the network topology involves defining the network's structure, while assigning jitter as a QoS metric represents packet delay variation. For each real-time application, calculate the jitter for each possible path using the following steps, Determine the available paths between the source and destination nodes, then calculate the jitter for each path using historical data or real-time measurements. Then, sort the paths in order of increasing jitter [19]. Select the path with the lowest jitter from the sorted list for each real-time application. Reserve network resources for the selected path, such as bandwidth and buffer space.For non-real-time applications, route traffic using the standard QoS routing algorithm. Weighted Fair Queueing (WFQ) is a commonly used QoS routing algorithm that assigns weights to traffic flows based on their requirements. It prioritizes flows, allocates bandwidth proportionally, and ensures fair resource sharing. Other QoS routing algorithms include SPF with QoS extensions, QoS-RP, and RSVP. Selection depends on network needs and desired QoS guarantees. Monitor network performance and adjust

RETRACTED ARTICLE

QoS routing parameters to optimize network performance. It is a simplified method for particular network topologies and QoS requirements. The actual implementation may require additional logic to handle network congestion, dynamically adjust routing paths, or handle failures in the network.

Figure 3 describes the packet arrival time. That is important in networked applications that require low latency and high throughput. Several techniques can be used to improve packet arrival time, including following factors.

Quality of Service (QoS): Implementing QoS mechanisms such as traffic shaping, prioritization, and scheduling can help to ensure that packets are delivered on time, even in congested networks. Network topology optimization: Optimizing the network topology can help to reduce the distance between the sender and receiver, thereby reducing the time it takes for packets to arrive. This can be done by using techniques such as network partitioning, load balancing, and routing optimization. Protocol optimization: Optimizing the network protocol can help to reduce the overhead associated with packet delivery, thereby improving packet arrival time. This can be done by using techniques such as packet compression, header compression, and protocol offloading [14]. Buffer management: Managing network buffers can help to reduce the delay associated with packet delivery by ensuring that packets are queued and processed efficiently. Efficient network buffer management strategies, such as proper sizing, intelligent algorithms, QoS differentiation, traffic engineering, congestion control, packet prioritization, and network monitoring, help reduce delays in packet delivery. This can be done by using techniques such as buffer sizing, congestion control, and flow control. Caching: Using caching techniques can help to reduce the time it takes for packets to arrive by storing frequently accessed data closer to the user. This can be done by using techniques such as content distribution networks (CDNs), proxy servers, and web caches. CDNs optimize content delivery by caching content on distributed servers globally, reducing latency and improving scalability. Proxy servers act as intermediaries, providing caching, filtering, and anonymity services for clients. Web caches store frequently accessed content at different levels, reducing retrieval time and alleviating network congestion. Buffer management: Managing network buffers can help to reduce the delay associated with packet delivery by ensuring that packets are queued and processed efficiently. This can be done by using techniques such as buffer sizing, congestion control, and flow control. Latency measurement and analysis: Measuring and analyzing network latency can help to identify bottlenecks and guide the selection of appropriate mitigation techniques

RETRACTED ARTICLE

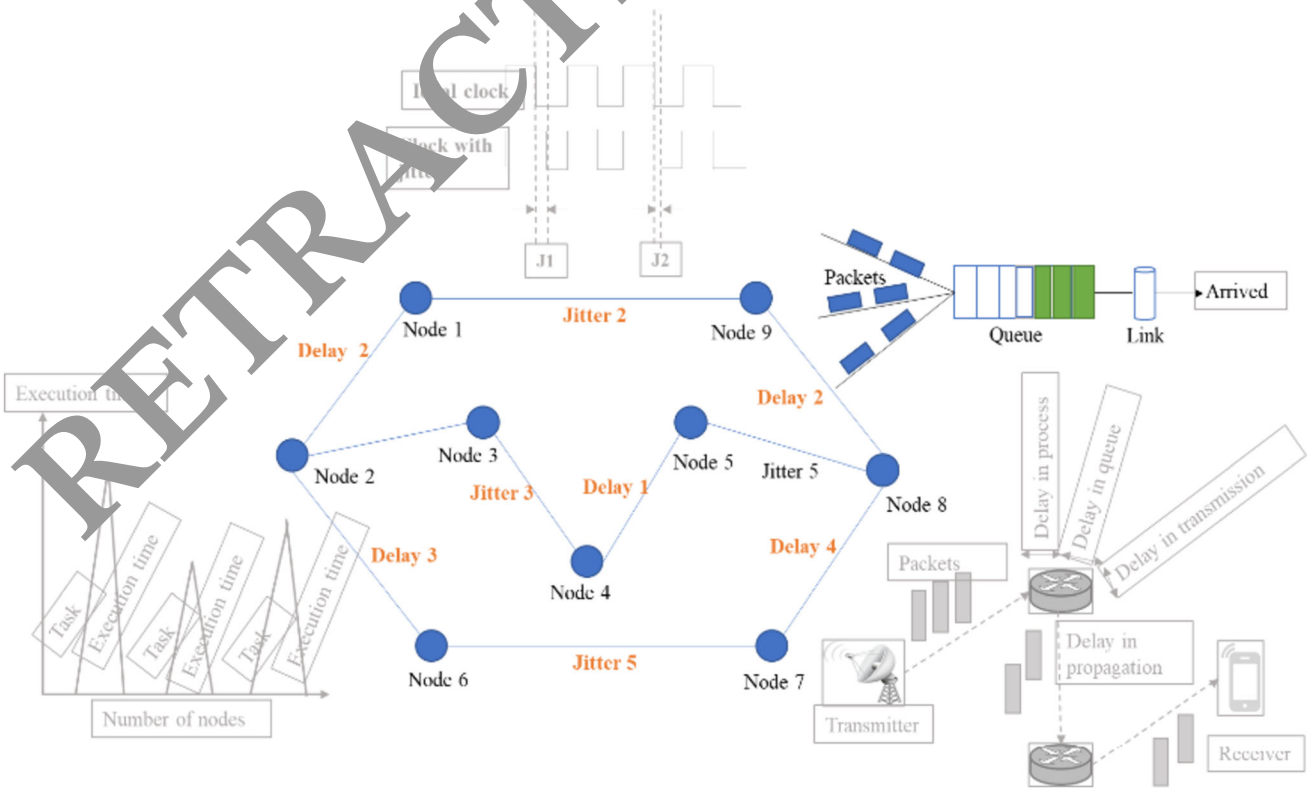



**Fig. 3** QoS routing with packet arrival time

[20]. By using these techniques, it is possible to improve packet arrival time, reduce latency, and enhance the overall performance of networked applications. To find packet arrival time between transmitter (n) and receiver (n-1) is:

$$\Delta c(n) = \frac{N}{f_s} + d(n) - d(n-1) = N\tau_s + d(n) - d(n-1) \tag{6}$$

where, 1/N is packet arrival time rate, $f_s$ is transmitter bit rate, and the packet arrival time between n and n-1.

$$\tau_s = \frac{E[\Delta c(n)]}{N} \tag{7}$$

where, E[d(n)-d(n-1)] = 0.

$$\widehat{\tau}_s(n) = \frac{1}{NL}\sum_{i=0}^{L-1}\Delta c(n-i) = \frac{\bar{\Delta}_{c(n)}}{N}. \tag{8}$$

where, L packets. These are packet arrival time averaging method. Packet arrival time can also be a factor that is considered in QoS routing. By analyzing the arrival time of packets, a QoS routing algorithm can identify the most efficient path for the packets to travel through the network by considering network conditions and QoS metrics, optimizing resources to meet desired QoS requirements. This can be particularly useful in situations where there are multiple possible routes for packets to take. The most efficient route for packets to travel through a network depends on several factors such as network topology, available resources, and current network conditions.

The packet arrival time factor can be used to calculate the cost of each edge in the network. For example, if the packet arrival time factor is high for an edge, it means that the edge has a high capacity in terms of data transmission. The efficiency of using packet arrival time as a factor to calculate the cost of each edge in the network depends on the specific routing algorithm and its implementation. Generally, the packet arrival time can be used to estimate the quality or congestion level of network links. Therefore, we can prioritize edges with higher packet arrival time factors to maximize the overall data transmission rate. The data transmission rate is calculated by dividing the amount of data sent by the time it takes to send it. The result is measured in bits per second (bps), which can be expressed as Kbps, Mbps, or Gbps. Techniques to improve packet arrival time include traffic shaping, QoS prioritization, network bandwidth optimization, error correction, routing optimization, network synchronization, packet prioritization, and network optimization. The algorithm uses Dijkstra's shortest path algorithm with the packet arrival time factor as the cost function. It maintains a Priority Queue of nodes to be visited, with the node with the lowest cost popping from the queue in each iteration [21]. The algorithm terminates when the Destination Node is reached and returns the path with the highest packet arrival time. This algorithm can be used in applications with essential QoS

**Algorithm for improving QoS routing using packet arrival time factor**

**Input:** Source_Node-S_N, Destination_Node-D_N, Neighbor_Node-Neigh_N

**function** QoS_Routing_Packet_Arrival_Time_Factor(S_N,D_N, Packet_Arrival_Time_Factor)

Path = [], Visited = [], Priority Queue = { S_N }, Cost = { S_N: 0}

  **while** Priority Queue is not empty:

    Current_Node = Pop_Node_With_Lowest_Cost(PriorityQueue, Cost)

    **if** Current_Node == D_N:

      return Path

    **If** Current_Node not Visited:

      Mark_As_Visited(Current_Node, Visited)

      for Neigh_N in Get_Neighbors(Current_Node):

        Cost_To_Neighbor =Calculate_Cost.Packet_Arrival_Time_Factor(Current_Node, Neigh_N, Packet_Arrival_Time_Factor)

        **if** Cost_To_Neighbor < Get_Current_Cost(Cost, Neigh_N):

          Update_Cost(Cost, Neigh_N, Cost_To_Neighbor)

          Set_Path(Path, Current_Node, Neigh_N)

          Add_Node_To_Priority_Queue(PriorityQueue, Neigh_N)

  **return** "Path not found."

RETRACTED ARTICLE

routing, such as online file sharing, cloud computing, and distributed databases to find the shortest path, optimizing data transfer, resource allocation, and response times. This algorithm efficiently finds the minimum cost path from a source node to all other nodes in a graph by iteratively selecting nodes with the smallest distances. It guarantees optimality for graphs with non-negative edge weights. In this pseudocode, the Pop_Node_With_Lowest_Cost function pops the node with the lowest cost from the priority queue, the Mark_As_Visited function marks a node as visited, the Get_Neighbors function returns the neighbors of a node, Calculate_Cost_Using_Execution_Time_Factor calculates the cost of an edge using the execution time factor, Get_Current_Cost returns the current cost of a node, Update_Cost updates the cost of a node, Set_Path sets the path from the source node to the current node and Add_Node_To_Priority_Queue adds a node to the priority queue [22].

Figure 4 describes the best techniques can be employed to improve execution time in software development. Here are some of the most commonly used techniques: Algorithm optimization: Optimizing the algorithms used in your code can have a significant impact on execution time. By improving the efficiency of the algorithm, you can reduce the number of operations required to complete a task and hence reduce the overall execution time. Code profiling: Code profiling helps you identify the parts of your code that take the most time to execute. By using a profiling tool, you can pinpoint the areas of your code that need optimization and make changes to improve the performance [23]. Parallelization: Parallelizing code involves breaking up a task into smaller subtasks and executing them concurrently on multiple processors. This can help improve execution time by leveraging the processing power of multiple cores. Memory management: Poor memory management can lead to slow execution times. By using memory efficiently, such as avoiding unnecessary allocation and deallocation of memory, you can reduce the overhead associated with memory management and improve execution time. Compiler optimization: Compiler optimization involves configuring the compiler to generate optimized machine code. This can help improve execution time by reducing the number of instructions required to perform a task [24].

### 3.1 Algorithm for improving QoS routing using execution time factor

The execution time factor can be used to calculate the cost of each edge in the network. For instance, if the execution time factor is high for an edge, the edge has a high cost in terms of time required for execution. Therefore, we can prioritize edges with lower execution time factors to minimize the overall execution time. To improve Quality of Service (QoS) routing using execution time factor, we can use the following algorithm:

**Input:** Source Node- S_N, Destination Node- D_N, Execution Time Factor

Initialize: Path = [], Visited = [], Priority Queue = { S_N }, Cost = {S_N: 0}

**Step1:**

**While** Priority Queue is not empty:

Pop the node with the lowest cost from the Priority Queue

**If** popped node = Destination Node:

Return Path

**If** node has not been visited:

Mark node as visited

**Step2:**

**For** each neighbor of the node:

Calculate the cost of the neighbor using the execution time factor

**If** the cost is lower than the current cost:

Update the cost

Set the Path to the current node + neighbor

Add the neighbor to the Priority Queue

**Return** Path not found

The algorithm maintains a Priority Queue of nodes to be visited, with the node with the lowest cost popping from the queue in each iteration. The algorithm terminates when the Destination Node is reached and returns the path with the lowest cost. This algorithm can be used in applications with essential QoS routing, such as real-time video streaming, online gaming, and teleconferencing.

Caching: Caching involves storing the results of expensive operations in memory so that they can be retrieved quickly in the future. This can help improve execution time by reducing the number of times an expensive operation needs to be performed. Use efficient data structures: Using the appropriate data structures for a given task can have a significant impact on execution time. For example, using a hash table to store key-value pairs can provide O (1) access time, which is much faster than using a linear search in an array.

Figure 5 describes the Improving delay, or reducing latency, is an important consideration in many technological applications, particularly those involving real-time processing and communication. Here are some techniques that can help reduce delay: Use a faster communication protocol: If you are experiencing delays due to slow

RETRACTED ARTICLE

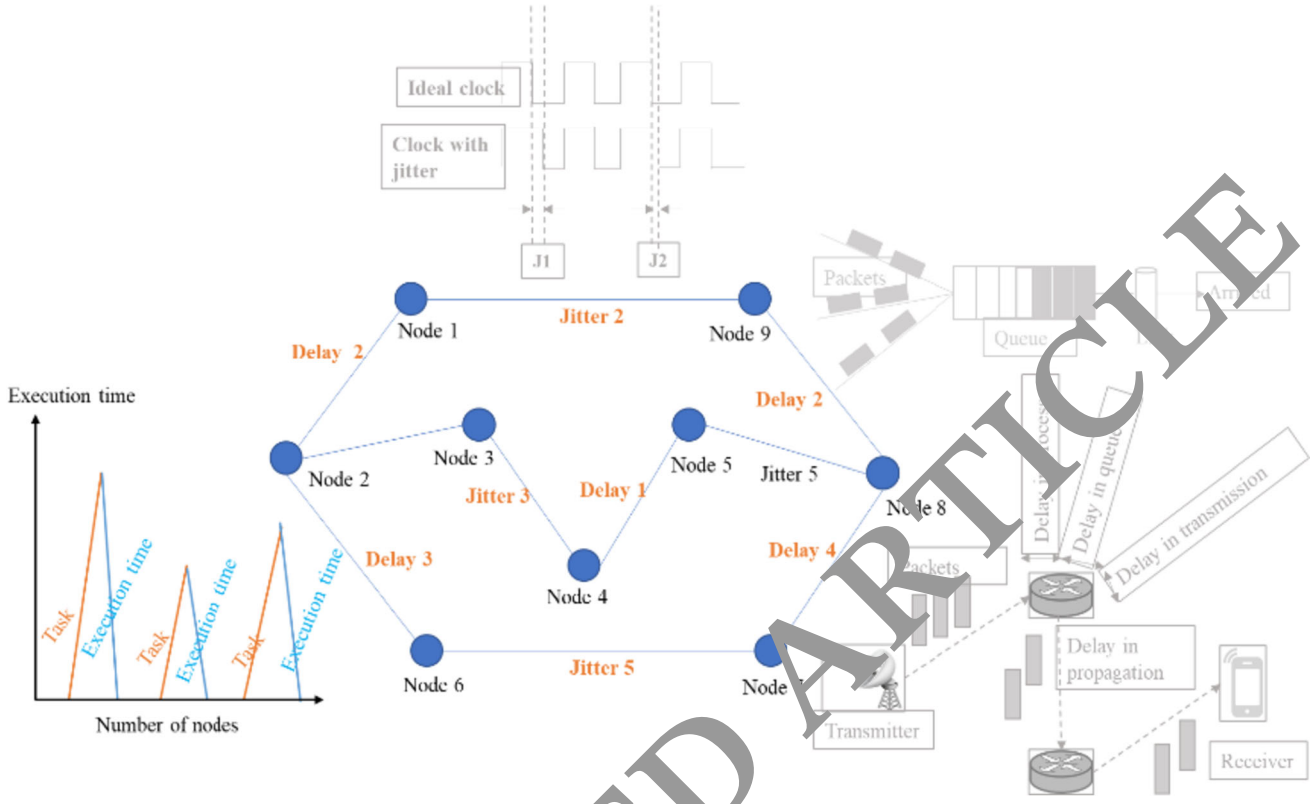



**Fig. 4** Processing flow for QoS routing with execution time

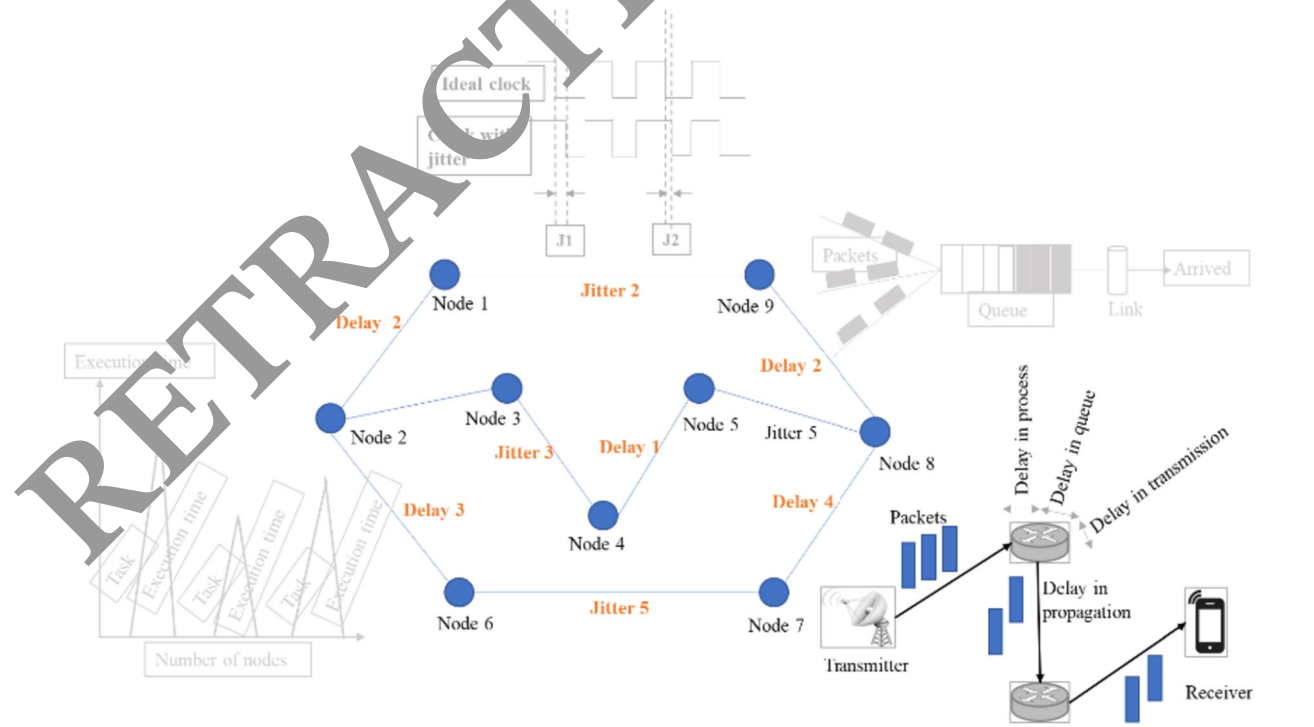



**Fig. 5** Processing flow for QoS routing with Delay

RETRACTED ARTICLE

communication, switching to a faster protocol can help. For example, using UDP instead of TCP/IP can reduce latency since UDP does not require the same level of error checking and correction. This reduced overhead of error checking and correction in UDP can help reduce latency. Optimize your network infrastructure: If you are experiencing delays due to network congestion, optimizing your network infrastructure can help. This might involve adding more bandwidth or optimizing your routing protocols to minimize hops and packet loss [25]. Implement caching: Caching involves storing frequently accessed data closer to the user or application, which can reduce the time required to retrieve the data. This is particularly useful in situations where the delay is caused by remote access to data sources. Use a content delivery network (CDN): A CDN is a distributed network of servers that are strategically located around the world. By using a CDN, content can be cached and delivered from a server that is closer to the user, reducing the time required to access the content. Use compression techniques: Compressing data can reduce the amount of time required to transmit it over a network. This is particularly useful in situations where bandwidth is limited. Optimize your application code: If your application code is inefficient, it can contribute to delay. Optimizing your code can help reduce the time required to execute it. Use hardware acceleration: Certain types of hardware, such as GPUs and FPGAs, can accelerate certain types of processing, such as video encoding and decoding. By offloading processing to specialized hardware, delay can be reduced. Use parallel processing: Parallel processing involves breaking a task down into smaller parts and executing those parts simultaneously on multiple processors. This can help reduce the time required to complete the task [24].

### 3.2 Algorithm for improving QoS routing using delay factor

Delay is when a packet travels from source to destination in a network. This includes the time to transmit the packet over the network and any processing delay at intermediate nodes. Delay is an essential metric for real-time applications such as voice and video, where excessive delay can result in degraded user experience.

**Input:** Source Node- S_N, Destination Node- D_N,

**Step1:**

**function** QoS_Routing_Delay_Factor(S_N, D_N, Delay_Factor):s

**Initialize** network topology and QoS metrics.

Path = [], Visited = [], PriorityQueue = { S_N }, Cost = { S_N: 0}

**while** PriorityQueue is not empty:

Current_Node = Pop_Node_With_Lowest_Cost(PriorityQueue, Cost)

**if** Current_Node == Destination_Node:

return Path

**if** Current_Node not in Visited:

Mark_As_Visited(Current_Node, Visited)

**for** Neighbor_Node in Get_Neighbors(Current_Node):

Cost_To_Neighbor = Calculate_Cost. Delay_Factor(Current_Node, Neigh_N, Delay_Factor)

**if** Cost_To_Neighbor < Get_Current_Cost(Cost, Neighbor_Node):

Update_Cost(Cost, Neighbor_Node, Cost_To_Neighbor)

Set_Path(Path, Current_Node, Neighbor_Node)

Add_Node_To_Priority_Queue(PriorityQueue, Neighbor_Node)

**return** Path not found

RETRACTED ARTICLE

Initially, collect the network data, such as delay and link utilization, from network devices using APIs or monitoring tools. Common network monitoring tools include network performance monitors, packet sniffers, network traffic analyzers, SNMP monitoring tools, flow analyzers, bandwidth monitoring tools, SIEM systems, network configuration management tools. Then prepare the data for machine learning by normalizing and transforming the features, such as delay, into a machine-readable format. Also, split the data into training and testing sets [26]. Train a machine learning model using the training data, and the model should be trained to predict the delay for each possible path given the network topology and QoS metrics. Evaluate the performance of the model using the testing data. Evaluate data performance using testing data through accuracy metrics, error metrics, cross-validation, performance plots, and domain-specific evaluation. These methods assess accuracy, measure discrepancies, validate models, and guide decision-making. Incorporate the trained model into the QoS routing algorithm. For each real-time application, use the model to predict the delay for each possible path and select the path with the lowest predicted delay. Reserve network resources for the selected path, such as bandwidth and buffer space. For non-real-time applications, route traffic using the standard QoS routing algorithm. Continuously monitor network performance and retrain the model periodically to improve its accuracy [1].

### 3.3 Novel fault tolerance against the QoS routing

To improve the factor of fault isolation using following methods. Fault isolation is the process of identifying the root cause of a fault or failure in a system, application, or component. Here are some techniques that can help improve fault isolation: Use logging and monitoring: Logging and monitoring tools can help track the behavior of a system or application over time, providing valuable data that can be used to identify faults and their causes. Logging and monitoring software is used to track, record, and analyse the behaviour and performance of systems, applications, networks, and infrastructure components. These tools collect data and generate logs, metrics, and alerts to provide visibility into the health, availability, and performance of various IT elements. Implement fault tolerance and redundancy: By building fault tolerance and redundancy into a system, you can reduce the impact of a fault or failure and make it easier to isolate the root cause of the problem. Use automated testing: Automated testing can help identify faults and failures early in the development process, making it easier fto isolate and resolve issues before they become critical [19]. Implement fail-safe mechanisms: Fail-safe mechanisms can help prevent faults and failures from cascading and causing further damage to a system or application. For example, a fail-safe mechanism might shut down a system or component automatically if it detects a fault. Use fault injection: Fault injection involves deliberately introducing faults or failures into a system to test its resilience and identify potential points of failure. This technique can be used during testing or in production to help isolate faults and improve fault tolerance. Implement distributed tracing: Distributed tracing is a technique that allows you to trace a request as it travels through a distributed system, making it easier to identify faults and their causes. Distributed tracing tools can help pinpoint the exact location of a fault and help with fault isolation. Use anomaly detection: Anomaly detection techniques can help identify unusual behavior in a system or application, which may be indicative of a fault or failure. By using anomaly detection, you can proactively identify faults before they cause significant damage and isolate their root cause [27].

### 3.4 Fault isolation estimators

The corresponding adoptive laws are used in the estimators of fault isolation.

$$\dot{\hat{x}} = A\hat{x}^s + \gamma(y,u) + L(\hat{y}^s - y) + \Omega^s\dot{\hat{\theta}}^s, \hat{x}^s(T_e) = 0 \tag{9}$$

$$\dot{\Omega}^s = \boldsymbol{A_O}\Omega^S + LF^S\Omega^S(T_e) = 0 \tag{10}$$

$$\hat{y}^s = C\hat{x}^s + F^S\hat{\theta}^s \tag{11}$$

$$\dot{\hat{\theta}}^s = \rho_{\ominus^s}\left\{\gamma^S\left(C\Omega^S + F^S\right)^T\epsilon_x^S\right\} \tag{12}$$

where, $T_e$ is fault detected at some time, $\ominus^s$ is hypersphere of radius, $\epsilon_x^S$ is output estimation error. These are the fault isolation estimators.

Figure 6 is gives details about Processing flow for fault tolerance improving factor. To improve the factor of fault detection using following methods. *Network Monitoring*: Network monitoring tools can be used to detect network faults by continuously monitoring the network for changes in network performance, traffic patterns, and other metrics. Monitoring tools can provide real-time alerts to IT staff when network issues arise, allowing them to quickly respond to and resolve issues. *Fault-tolerant Design*: Fault-tolerant network design can help to minimize the impact of network faults by providing redundant network paths and components. For example, redundant power supplies, network cards, and network links can be deployed to ensure that if one component fails, there is a backup component to take over [28]. *Regular Network Testing*: Regular network testing can help to identify potential network issues before they become critical faults. Testing can include traffic analysis, load testing, and stress testing to ensure that the

RETRACTED ARTICLE

network can handle the expected traffic and performance requirements. *Proactive Network Maintenance*: Regular maintenance and upgrades can help to prevent network faults before they occur. Firmware upgrades, hardware upgrades, and other maintenance tasks can help to ensure that the network is running smoothly and that potential issues are addressed before they become faults. *Automated Fault Detection*: Automated fault detection systems can be used to monitor network performance and detect issues in real time. Automated fault detection systems effectively monitor network performance, detect problems in real-time, enable proactive issue identification, rapid problem resolution, network optimization, scalability, efficiency, and data-driven decision making, with human expertise for comprehensive network management. These systems can provide early warning alerts to IT staff, allowing them to respond quickly and prevent downtime or other network disruptions. *Network Segmentation*: Network segmentation can help to limit the impact of network faults by isolating affected areas of the network. By breaking the network into smaller, more manageable segments, faults can be contained, and their impact can be minimized [19].

To improve the factor of data redundancy using following methods. *RAID (Redundant Array of Independent Disks)*: RAID is a technology that uses multiple hard drives to store data redundantly. There are different RAID levels, each offering different levels of redundancy and performance. RAID can provide fault tolerance and improved performance for data storage. It is widely used in servers, storage systems, and high-performance computing environments for increased storage capacity and data protection. It utilizes data striping to improve access speeds and offers different levels of redundancy to protect against drive failures. *Replication*: Replication involves making copies of data and storing them on multiple servers or locations. This can be done either synchronously or asynchronously. Synchronous replication ensures that all copies are identical and up-to-date, while asynchronous replication may introduce some delay between copies. Replication can provide redundancy and improve the availability of data. *Backup and Recovery*: Backup and recovery systems can be used to create and store copies of data, which can be used to recover lost or corrupted data. Regular backups can ensure that data is always recoverable, even if the original data is lost or damaged. *Cloud Storage*: Cloud storage providers often offer data redundancy features as part of their service. They store data in multiple locations and provide automated backups to ensure that data is always available. *Data Archiving*: Data archiving involves moving data that is no longer frequently accessed to long-term storage. This can help to free up space on primary storage systems and reduce the risk of data loss. Disaster *Recovery Planning*: Disaster recovery planning involves developing a plan for recovering data in the event of a disaster, such as a natural disaster, cyberattack, or hardware failure. This plan can include backup and recovery systems, redundant hardware and storage systems, and other measures to ensure that data can be recovered quickly and effectively [4].

To improve the factor of robustness using following methods. *Defensive Programming*: Defensive programming is a technique that involves anticipating potential errors and handling them in a way that minimizes their impact. This can include using error-handling mechanisms, input validation, and defensive coding practices. *Failover Mechanisms*: Failover mechanisms provide a way for systems to continue functioning even if one or more components fail. This can involve deploying redundant hardware or software components, as well as designing failover mechanisms to automatically switch to backup components if failure occurs. *Load Balancing*: Load balancing distributes workloads across multiple servers or systems, ensuring that no single system is overwhelmed with too much traffic or work. Load balancing can improve system performance and availability by distributing workloads and reducing the risk of downtime due to overloaded systems. *Regular System Testing*: Regular system testing can help to identify potential issues before they become critical. This can include load testing, stress testing, and other types of testing to ensure that systems can handle expected traffic and performance requirements. *Security Measures*: Security measures can help to protect systems from cyber threats and attacks. This can include using firewalls, intrusion detection systems, and other security mechanisms to prevent unauthorized access and protect against malware and other threats. *Disaster Recovery Planning*: Disaster recovery planning involves developing a plan for recovering systems and data in the event of a disaster, such as a natural disaster, cyberattack, or hardware failure. This plan can include backup and recovery systems, redundant hardware and storage systems, and other measures to ensure that systems can be recovered quickly and effectively [29].

### 3.5 Common factor in (CRAFT) protocol for improving the "novel QoS routing" and "fault tolerance"

The Coalescing novel QoS Routing and fault tolerance (CRAFT) created with improving the QoS routing and Fault tolerance. The QoS routing and Fault tolerance has similar factors to improve.

- Jitter against the fault isolation
- Packet arrival time against the data redundancy
- Execution time against the fault detection

RETRACTED ARTICLE

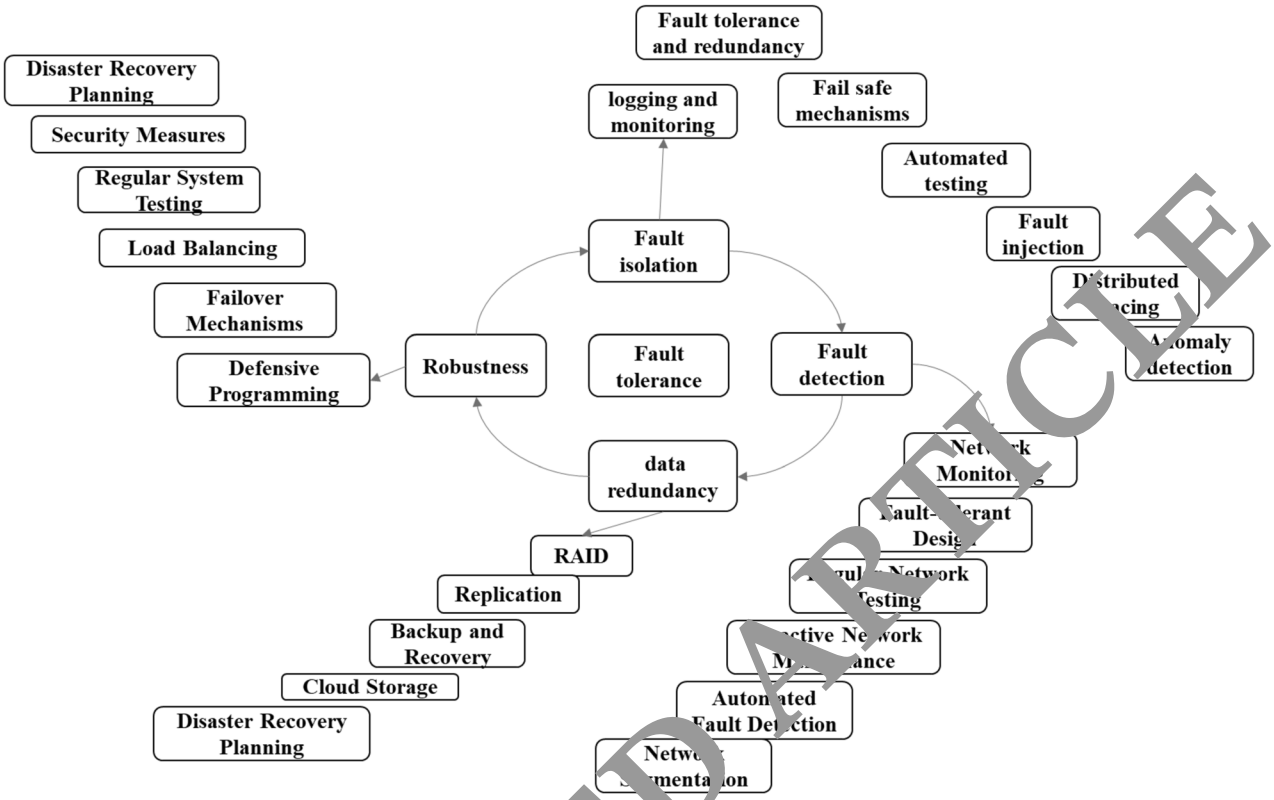



**Fig. 6** Processing flow for fault tolerance improving factor

- Delay against the robustness

Jitter can be an indicator of faults or failures within a system. By monitoring the amount of jitter in a system, engineers may be able to identify potential problems and isolate the source of the issue. For example, if the jitter in a digital communication system increases suddenly and significantly, this may indicate a fault in the clock or timing circuitry. By isolating this fault, engineers can work to repair or replace the affected components and restore the system to its proper functioning. In this way, jitter and fault isolation are closely related. Jitter can be both a symptom of a fault and a tool for identifying the source of the fault. By monitoring and analyzing jitter, engineers can improve the reliability and performance of electronic systems, and ensure that faults are quickly and accurately isolated and addressed [27].

The relationship between packet arrival time and data redundancy is complex, as increasing data redundancy can both improve and worsen packet arrival times. On the one hand, adding redundancy to a transmission can increase the amount of data that needs to be transmitted, which can increase the overall time it takes for packets to arrive. This is because each packet must carry additional information that must be transmitted and received. On the other hand, data redundancy can also improve packet arrival times by helping to ensure the accuracy and integrity of the data being transmitted. By including error-correcting codes and other forms of redundancy, data can be checked and corrected for errors before it is transmitted, reducing the need for retransmissions and improving overall network efficiency. In general, the relationship between packet arrival time and data redundancy will depend on a variety of factors, including the specific application, the network environment, and the amount of redundancy being used. By carefully balancing the need for data redundancy with the need for efficient and timely packet transmission, network engineers can optimize network performance and ensure reliable data transmission [30].

The relationship between execution time and fault detection is a trade-off. If the software system is designed to prioritize execution time, it may sacrifice some of its fault detection capabilities. Conversely, if the software system prioritizes fault detection, it may slow down its execution time. However, it is possible to strike a balance between execution time and fault detection by implementing efficient and effective error handling mechanisms within the software system. This can involve incorporating error detection and correction algorithms, implementing exception handling routines, and conducting thorough testing and debugging procedures to identify and fix any errors that may arise. Overall, while execution time and

RETRACTED ARTICLE

fault detection are two distinct aspects of software engineering, they are interrelated and should be considered in the design and development of any software system to ensure its optimal performance and reliability. In some cases, increasing robustness can lead to increased delay. For example, adding redundant components to a system can increase its robustness, but may also increase the time it takes to process information or make decisions [15]. Similarly, increasing the complexity of a control system to improve robustness may also increase its delay. On the other hand, reducing delay can sometimes compromise robustness. For example, reducing the delay in a control system may require sacrificing some robustness in order to respond more quickly to disturbances. Therefore, it's important to carefully balance the trade-off between delay and robustness depending on the specific application and requirements of the system. These factors find Coalescing novel QoS Routing and fault tolerance (CRAFT) protocol against the efficient routing [30].

## 4 Results and discussion

In this section, we use the CRAFT protocol to estimate the Coalescing of novel QoS Routing and fault tolerance, improving QoS parameters in the ad-hoc wireless network. To optimize QoS parameters in ad-hoc wireless network focus on routing optimization, power management, channel allocation, MAC protocol optimization, quality aware routing metrics, etc., In the first step, we analyze the novel QoS routing using factors such as Jitter, packet arrival time, Execution time, and delay. In addition, the novel fault tolerance is based on fault isolation, fault detection, data redundancy, and robustness. Finally, forming Common Factor in (CRAFT) protocol for Improving the "Novel QoS Routing" and "Fault Tolerance based on the Jitter against fault isolation, packet arrival time against data redundancy, Execution time against fault detection, and delay against robustness. Our proposed CRAFT protocol is compared with various existing methods in terms of the following parameters: Jitter, packet arrival time, Execution time, delay, fault isolation, fault detection ratio, data redundancy, and robustness. Our proposed CRAFT protocol is estimated in the simulation software of network simulator -2(NS2). There are 100–400 nodes in the network with a size of 1600 m × 1600 m, and the other variables are displayed in Table 1.

Our proposed CRAFT protocol is compared with the existing OSPF-TE (Open Shortest Path First—Traffic Engineering), RSVP-TE (Resource Reservation Protocol-Traffic Engineering), and PCEP (Path Computation Element Protocol) protocol, where the OSPF-TE and RSVP-TE are traditional QoS-based routing protocols, the PCEP is a newer protocol and proposed CRAFT protocol is a novel protocol that aims to improve performance in congested traffic conditions.

The experimental measures are explained as follows: The Jitter is a measure of the variation in the timing of digital signal edges, which can result in errors in data transmission. It is typically measured in units of nanoseconds (ns). On the other hand, frequency measures the number of cycles per second of a periodic signal, typically measured in Hertz (Hz) or Gigahertz (GHz) units. The relationship between the frequency and jitter is shown in Table 2.

Figure 7 shows a measure jitter based on the increasing frequency of GHz. When the frequency has increased, the jitter also increased linearly, and when the frequency is 6 GHz, the jitter becomes suddenly higher than one another; after that, it is slightly decreased to reach its approximately similar jitter value. At 6HZ, the existing OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol has a jitter of 8.4 ns, 5.5 ns, 10.3.5 ns, and 6.3 ns, respectively. At the initial 1 GHz frequency, the jitter values as 4 ns, 3 ns, 3.7 ns, and 3.5 ns for OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol correspondingly, similarly, at the end of 10 GHz frequency, the jitter as, 3.3 ns, 3.3 ns,3.2 ns, and 3.1 ns OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol respectively, from this, our proposed CRAFT protocol ensures the better performance than the other existing method in terms of jitter.

*Packet arrival time*: The Packet arrival time is an essential metric in evaluating the performance of a communication network. Packet arrival time refers to the time it takes for a packet of data to reach its destination in a communication network, and it is calculated for various routing protocols, as shown in Table 3. In a packet-switched network, data is divided into small packets transmitted individually.

Each packet contains the necessary information to route it through the network to its destination. Packet arrival time can be affected by several factors, such as network congestion, transmission errors, queuing delays, and the efficiency of the routing protocols used in the network. A low packet arrival time means that data is transmitted quickly and efficiently through the network, which can lead to improved overall network performance. Conversely, a high packet arrival time can indicate issues with the network that need to be addressed, such as congestion or inefficient routing protocols. High packet arrival times indicate potential network problems such as congestion, bottlenecks, routing issues, QoS problems, equipment performance issues, and SLA violations. Monitoring packet delay helps identify and troubleshoot network issues to optimize performance and improve the user experience. Network

RETRACTED ARTICLE

**Table 1** Simulation metrics

| Simulation parameter | Values |
|---|---|
| Network size | 1600 m × 1600 m, |
| Node counts | 400 |
| Node energy | 5 J |
| Data rate/time | 300 kb/5 min |
| data packet size | 128 bytes |
| the capacity of Queue packets | 40 |

administrators and engineers aim to minimize packet arrival time by optimizing network performance through load balancing, congestion control, and Quality of Service (QoS) management.

Figure 8 depicts the packet arrival time in a network for different routing protocols like CRAFT, OSPF-TE, RSVP-TE, and PCEP based on the number of nodes. The packet arrival time of OSPF-TE, RSVP-TE, and PCEP is 20.34%, 15.34%, and 12.43%, respectively, lesser than our proposed CRAFT protocol. In CRAFT, packet arrival time may increase with the number of nodes in the network because it uses congestion control mechanisms to avoid congestion and packet loss. As the number of nodes increases, the probability of congestion and packet loss also increases, resulting in longer packet arrival times. However, the packet arrival time may be affected by the complexity of the path and the network topology. Hence our proposed method overperforms the existing methods.

*Execution time*: Execution time (ms) refers to the time it takes for a particular task or process to be completed by a system, typically measured in milliseconds (ms). In the context of a communication network, improving execution time can lead to faster transmission of data and reduced packet arrival times. It is displayed in Table 4 with various protocols.

Execution time is an essential metric in evaluating the performance of a system or application. A low execution time indicates that a task or process is being completed quickly and efficiently, which can lead to improved overall system performance and better Quality of Service (QoS) for the end user. This can be achieved through various optimization techniques, such as improving the efficiency of routing protocols, reducing packet collisions and retransmissions, and optimizing the use of network resources.

Figure 9 represents the different protocols over the routing path of data transmission through an increasing count of nodes based on their execution time in milliseconds. Figure 3 shows that the OSPF-TE has a higher execution time than the RSVP-TE, PCEP, and proposed CRAFT protocol. The RSVP-TE has a higher execution time than the PCEP and proposed CRAFT protocol. The PCEP has a higher execution time than our proposed CRAFT protocol. From this result, our proposed execution time is less than other existing protocols. When the execution time is reduced, it improves overall network performance. Quality of Service (QoS) can be improved, leading to higher reliability, availability, and performance for network users.

*End-to-end Delay*: Delay is a crucial metric for evaluating QoS-based routing protocols, and its approximately analyzed based on the increasing count of nodes, as shown in Table 5.

When traffic is congested, if the traffic rate of unusual events at intermediate nodes surpasses the CRAFT threshold, the unusual event packets are routed along the shortest path, while routine data packets are forwarded via random alternative paths. This is done to ensure that the network can handle the increased load and maintain a satisfactory level of performance.

The proposed protocol's delay results are presented in Fig. 10, which displays the delay for each routing protocol

**Table 2** Jitter of various networking protocols

| Frequency (GHz) | Jitter (ns) | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 1 | 4 | 3 | 3.7 | 3.5 |
| 2 | 4.5 | 3.6 | 3.9 | 4.1 |
| 3 | 5 | 4.1 | 4.3 | 4.9 |
| 4 | 5.7 | 4.6 | 5.4 | 5.2 |
| 5 | 6.8 | 4.9 | 8.4 | 5.9 |
| 6 | 8.4 | 5.5 | 10.3 | 6.3 |
| 7 | 4.8 | 4.1 | 5.3 | 4.4 |
| 8 | 4.1 | 3.8 | 4.6 | 4.1 |
| 9 | 3.8 | 3.5 | 3.9 | 3.6 |
| 10 | 3.3 | 3.3 | 3.2 | 3.1 |

RETRACTED ARTICLE

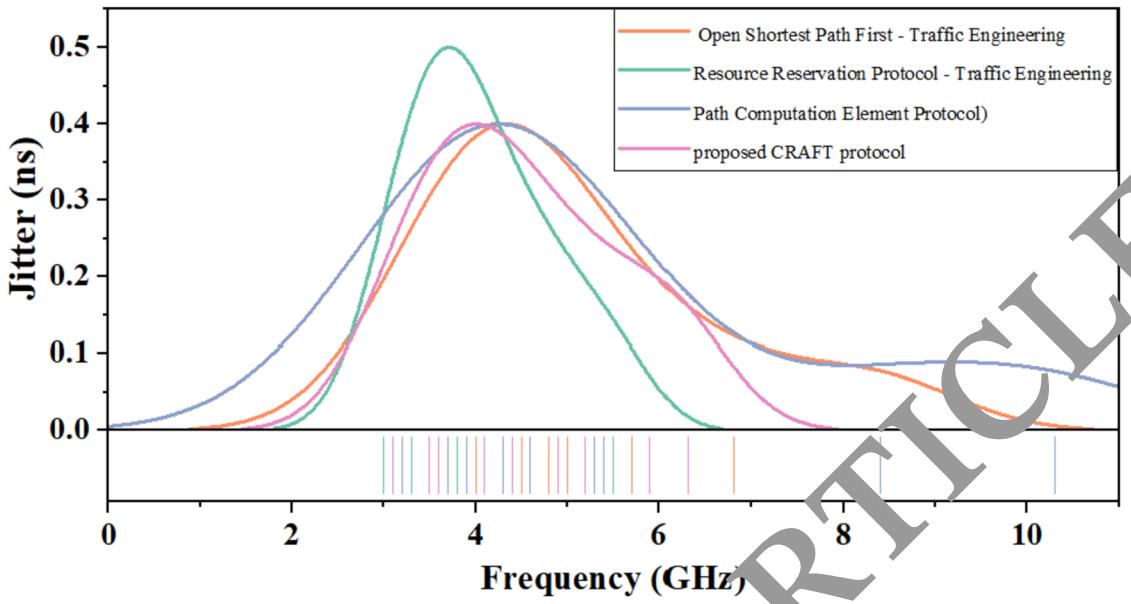


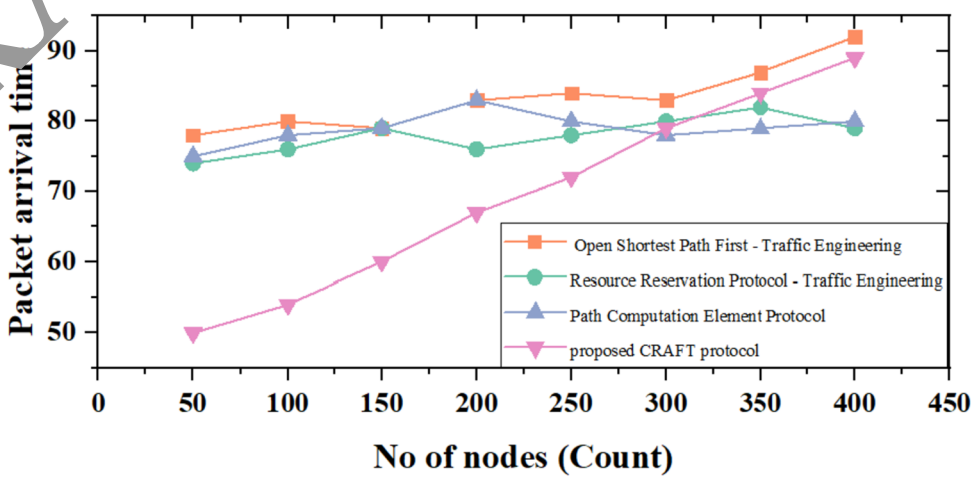

**Fig. 7** Performance Comparison of Network Routing Protocols with jitter factor

**Table 3** Packet arrival time of various networking protocols

| No of nodes (Count) | Packet arrival time | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 50 | 78 | 74 | 75 | 50 |
| 100 | 80 | 76 | 78 | 54 |
| 150 | [illegible] | 79 | 79 | 60 |
| 200 | 83 | 76 | 83 | 67 |
| 250 | 84 | 78 | 80 | 72 |
| 300 | 83 | 80 | 78 | 79 |
| 350 | 87 | 82 | 79 | 84 |
| 400 | 92 | 79 | 80 | 89 |

RETRACTED ARTICLE

**Fig. 8** Performance Comparison of Network Routing Protocol with packet arrival time factor




as the node count increases. Evaluate the delay for different node counts as 50, 100,150, 200,250, 300, 350and 400. For each setup, we have varied the simulation time. The exact amount of data packets transmitted over the network with 50 nodes, the OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol has a delay of 0.041, 0.039, 0.035, and

0.032, respectively. Network with 300 counts of nodes, each protocol has a delay of 0.072, 0.070, 0.071, and 0.057 for OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol, respectively. From this result, our proposed method has lesser delay than the other existing protocols.

Fault isolation is critical in troubleshooting and repairing complex systems, enabling technicians to focus on resolving the underlying issues. Fault isolation identifies the root cause of a problem or fault in a system that is measured for various protocols based on the node's increasing count, as shown in Table 6. It involves narrowing down the possible causes of the fault to a specific component or subsystem responsible for the problem.

CRAFT protocol uses a distributed approach that allows for fault isolation at the network edge, reducing the impact of faults on the overall network performance. Figure 11 shows that as the network grows, fault isolation becomes more challenging, and protocol CRAFT can offer better fault isolation and recovery capabilities than PCEP, OSPF-TE, and RSVP-TE. The graph shows the performance of four different routing protocols: OSPF-TE, RSVP-TE, PCEP, and CRAFT, in terms of fault isolation as the number of nodes in the network increases. As the number of nodes increases, the fault isolation capability of OSPF-TE and RSVP-TE tends to decrease. PCEP shows better fault isolation as the number of nodes increases. The proposed CRAFT protocol shows the best performance in terms of fault isolation, even as the number of nodes increases.

Fault Detection (FD) is a statistical used to evaluate a system's performance, and it is displayed in Table 7. It represents the ratio of the number of faults detected by the system to the total number of faults that occurred during a given period.

Figure 12 shows the fault detection performance of four routing protocols, OSPF-TE, RSVP-TE, PCEP, and the proposed CRAFT protocol, as the number of nodes increases. As the number of nodes in the network increases, the fault detection performance of OSPF-TE and RSVP-TE deteriorates. The PCEP, on the other hand, performs better than OSPF-TE and RSVP-TE due to its larger networks. However, its fault detection performance still deteriorates as the network size increases. The proposed CRAFT protocol outperforms all three protocols, particularly in larger networks. CRAFT uses localized detection mechanisms to improve fault detection in congested traffic conditions. It achieves this by dynamically adjusting the routing paths of packets based on the traffic load and network conditions, reducing the signaling overhead and improving the network's fault tolerance. Figure 6 shows that the proposed CRAFT protocol performs better fault detection than traditional QoS-based routing protocols in congested traffic conditions, particularly in larger networks.

Data redundancy refers to duplicating data within a database or a system. It occurs when the same piece of data is stored multiple times in different places, either within the same database or across multiple databases. While data redundancy leads to inconsistencies and inaccuracies in data, increases storage requirements, and complicates data maintenance and updates. In addition, it can make it challenging to maintain data integrity and ensure data consistency across different parts of the system, which is captured in Table 8. To ensure data consistency, methods such as atomicity, locking, concurrency control, data validation, backup and recovery, synchronization, versioning, and auditing are used.

In Fig. 13, the number of nodes in a network can impact the redundancy and scalability of various routing protocols, including OSPF-TE, RSVP-TE, PCEP, and the proposed CRAFT protocol. As the number of nodes in a network increases, the complexity of the path computation process in existing OSPF-TE, The RSVP-TE, and PCEP can also increase. This can lead to potential issues with scalability and redundancy. The proposed CRAFT protocol is designed to provide efficient traffic engineering in large networks with reduced processing and memory requirements. CRAFT protocol helps to improve scalability and redundancy in large networks with a high number of nodes. In summary, As the number of nodes in a network increases, the complexity of path computation and

**Table 4** Execution time of various networking protocols

| No of nodes (Count) | Execution time (ms) | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 50 | 0.94 | 0.85 | 0.82 | 0.74 |
| 100 | 0.96 | 0.87 | 0.84 | 0.79 |
| 150 | 0.93 | 0.85 | 0.86 | 0.76 |
| 200 | 0.96 | 0.89 | 0.85 | 0.73 |
| 250 | 0.95 | 0.86 | 0.84 | 0.75 |
| 300 | 0.94 | 0.87 | 0.83 | 0.79 |
| 350 | 0.97 | 0.89 | 0.85 | 0.74 |
| 400 | 0.92 | 0.92 | 0.87 | 0.77 |

RETRACTED ARTICLE

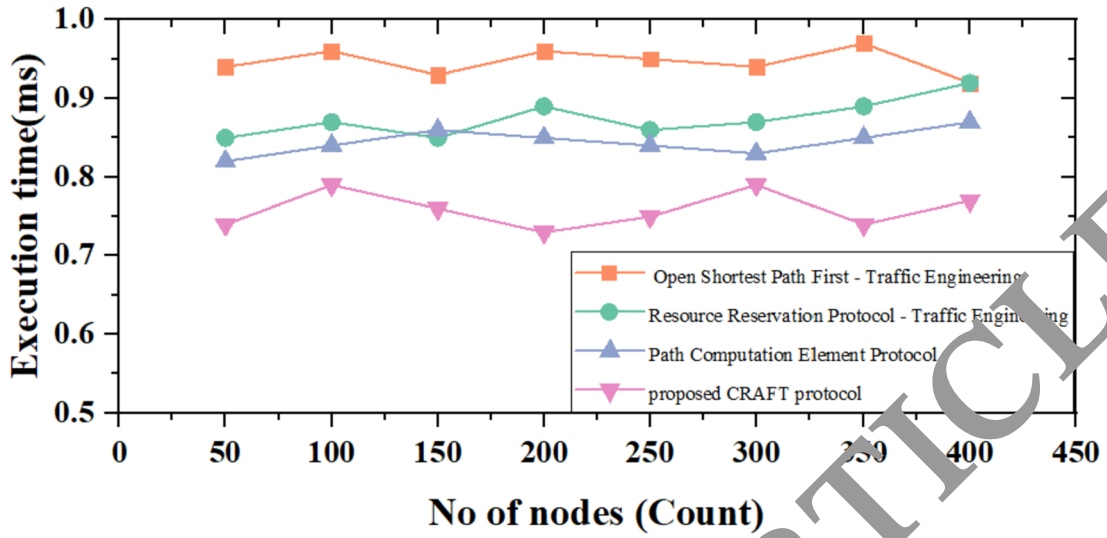



**Fig. 9** Performance Comparison of Network Routing Protocols with execution time factor

**Table 5** Delay of various networking protocols

| No of nodes (Count) | Delay | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 50 | 0.041 | 0.039 | 0.035 | 0.032 |
| 100 | 0.049 | 0.049 | 0.047 | 0.028 |
| 150 | 0.055 | 0.047 | 0.053 | 0.042 |
| 200 | 0.061 | 0.058 | 0.060 | 0.049 |
| 250 | 0.069 | 0.064 | 0.065 | 0.052 |
| 300 | 0.072 | 0.070 | 0.071 | 0.057 |
| 350 | 0.079 | 0.076 | 0.074 | 0.062 |
| 400 | 0.087 | 0.082 | 0.078 | 0.067 |

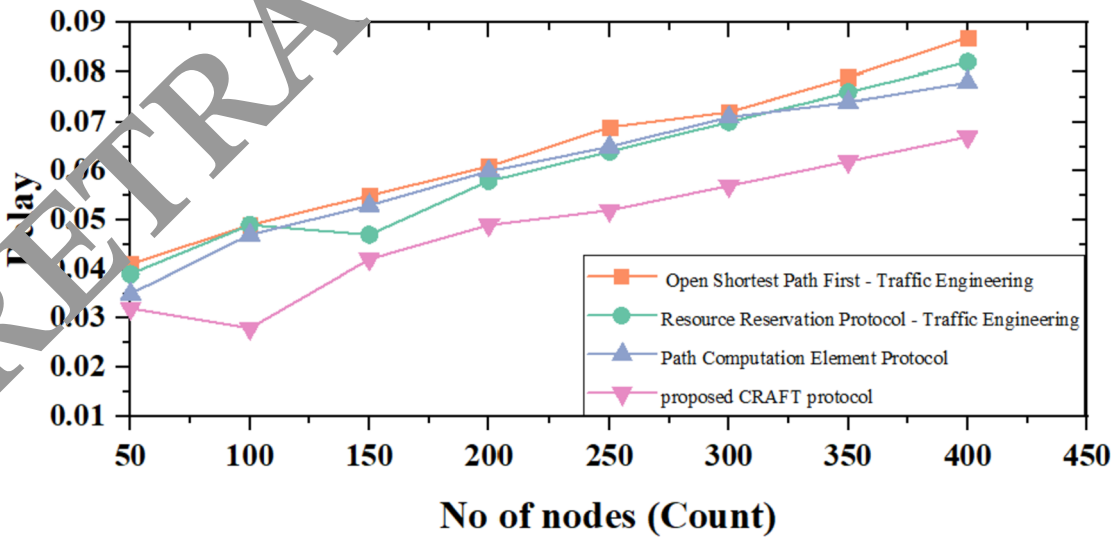



**Fig. 10** Performance comparison of network routing protocols with delay factor

signaling can also increase, potentially leading to issues with scalability and redundancy. The proposed CRAFT protocol is designed to provide efficient traffic engineering in large networks with reduced processing and memory

RETRACTED ARTICLE

**Table 6** Fault Isolation of various networking protocols

| No of nodes (Count) | Fault isolation | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 50 | 75 | 84 | 80 | 82 |
| 100 | 79 | 82 | 83 | 85 |
| 150 | 74 | 84 | 85 | 88 |
| 200 | 76 | 81 | 82 | 84 |
| 250 | 78 | 80 | 86 | 89 |
| 300 | 74 | 82 | 84 | 90 |
| 350 | 78 | 84 | 86 | 92 |
| 400 | 79 | 83 | 82 | 95 |

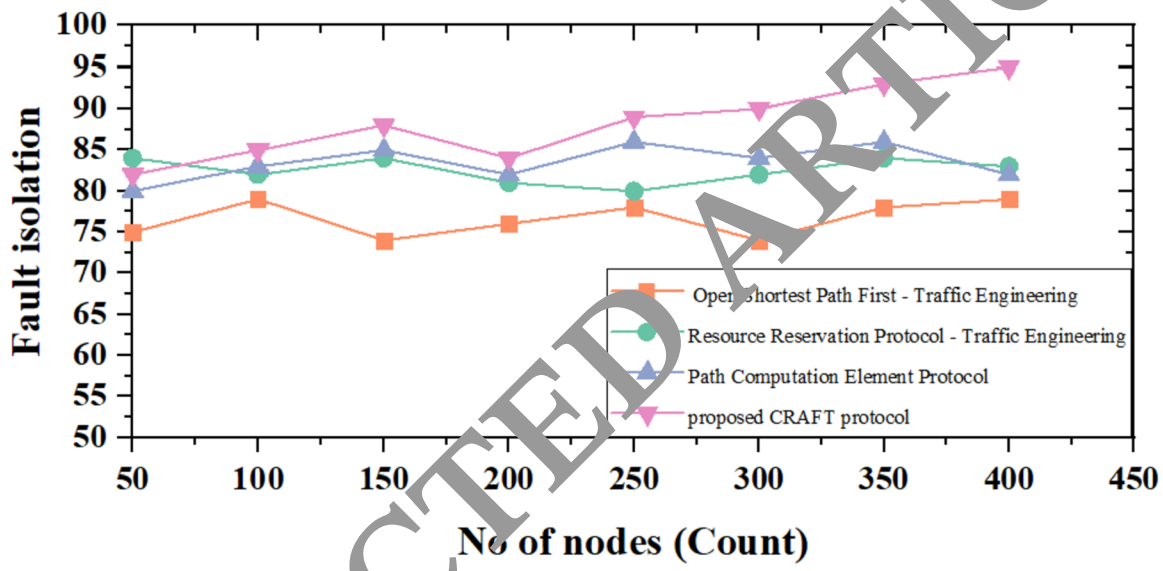



**Fig. 11** Performance Comparison of Network Routing Protocols with fault isolation factor

requirements, which can help to improve scalability and redundancy.

Robustness is the ability of a system to maintain its functionality and performance under adverse or changing conditions. In the context of network protocols, robustness refers to the ability of a protocol to continue operating effectively even in the face of various types of failures or unexpected events, such as network congestion, link failures, and node failures. Also, it is expressed numerically, that shown in Table 9.

Figure 14 depicts the robustness value for various simulation times (100–500 s) and different protocols, as proposed CRAFT protocol with the existing OSPF-TE, RSVP-TE, and PCEP. The existing protocols have increased

**Table 7** Fault detection of various networking protocols

| No of nodes (Count) | Fault Detection | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 50 | 78 | 80 | 79 | 82 |
| 100 | 80 | 76 | 82 | 85 |
| 150 | 83 | 79 | 78 | 87 |
| 200 | 85 | 82 | 82 | 89 |
| 250 | 78 | 83 | 84 | 91 |
| 300 | 73 | 85 | 86 | 94 |
| 350 | 76 | 87 | 84 | 90 |
| 400 | 79 | 85 | 83 | 93 |

RETRACTED ARTICLE

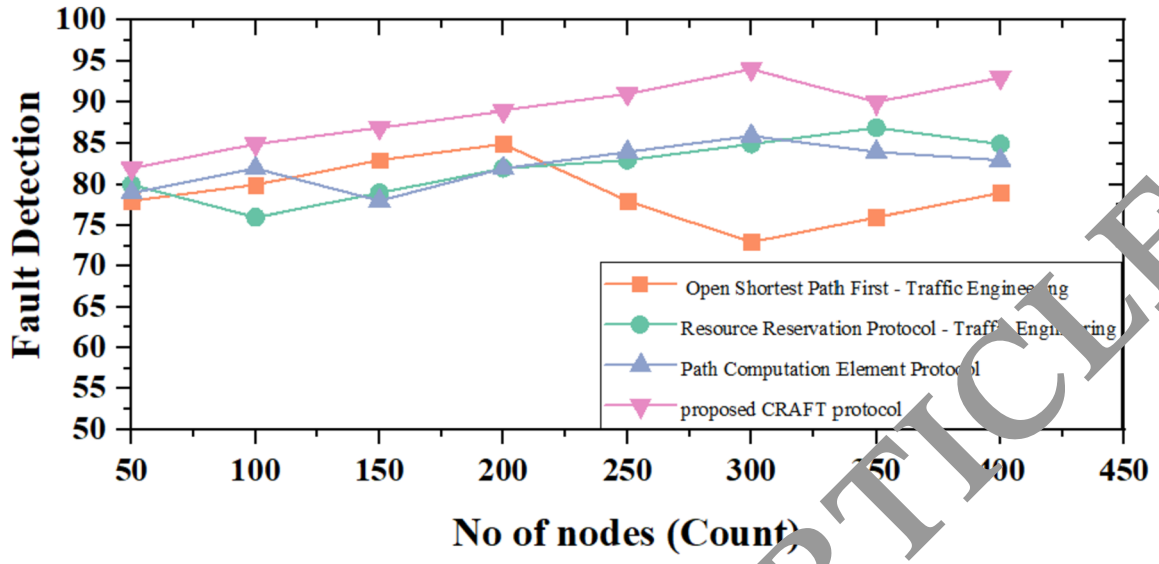



**Fig. 12** Performance Comparison of Network Routing Protocols with fault detection factor

**Table 8** Data redundancy rate of various networking protocols

| No of nodes (Count) | Data Redundancy Rate | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 50 | 0.4 | 0.6 | 0.8 | 10 |
| 100 | 1.6 | 2.7 | 3.4 | 15 |
| 150 | 3.5 | 4.3 | 5.2 | 19 |
| 200 | 5.3 | 5.5 | 7.6 | 24 |
| 250 | 7.1 | 7.2 | 9.3 | 29 |
| 300 | 8.5 | 9.4 | 11.8 | 32 |
| 350 | 9.5 | 11.3 | 13.1 | 38 |
| 400 | 11 | 13 | 15 | 45 |

signaling overhead that impacts network performance, but our proposed CRAFT is used to optimize network performance. As the number of nodes in a network increases, the complexity of the task also increases. At the simulation time of 20 ms, The OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol have the robustness of 0.79, 0.75, 0.82, and 0.86, respectively. At the end of 100 ms, the methods have the robustness of 0.85, 0.80, 0.84, and 0.95 for OSPF-TE, RSVP-TE, PCEP, and proposed CRAFT protocol correspondingly. However, CRAFT allows us to adapt to changing network conditions and optimize performance even in large networks. In summary, the number of nodes in a network can impact the robustness of network protocols. While OSPF-TE, RSVP-TE, and PCEP may face scalability issues as the network grows, CRAFT allows it to adapt to changing network conditions and optimize performance even in large networks.

## 5 Conclusion

The proposed system of CRAFT is used for the analysis and the improvements in this study is maintained for the delivery of the packets in the router successfully and without elimination of the messages is done. The shortest and the reliable way of transferring the data and the efficiency of the protocol sharing has been enabled here for the analysis. This analysis may have happened anywhere and to check the reliability and then the data can be send in the router with the following methods without traffic. CRAFT is used in finding the packet delivery, routing, and the execution time. This can be calculated in the networks using this CRAFT method. QoS routing helps in routing the information in the networks and transfer the data from one hub to the another. These transfer of the data establish the data from one place to the another the data may be struck in some network places and this method helps in

RETRACTED ARTICLE

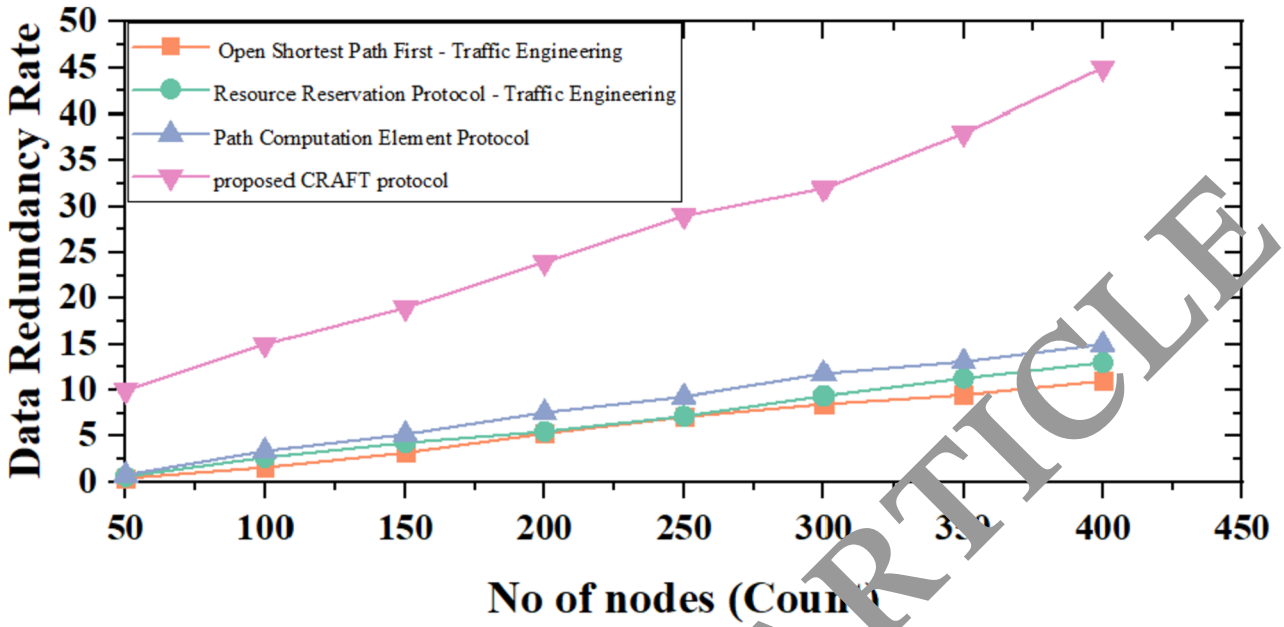



**Fig. 13** Performance comparison of network routing protocols with data redundancy rate

**Table 9** Robustness of various networking protocols

| Simulation Time (ms) | Robustness | | | |
|---|---|---|---|---|
| | OSPF-TE | RSVP-TE | PCEP | Proposed CRAFT protocol |
| 20 | 0.79 | 0.75 | 0.82 | 0.86 |
| 40 | 82 | 0.78 | 0.85 | 0.89 |
| 60 | 0.84 | 0.76 | 0.83 | 0.92 |
| 80 | 0.83 | 0.78 | 0.81 | 0.94 |
| 100 | 0.85 | 0.80 | 0.84 | 0.95 |

this. Based on the fraud tolerant the duplicate data has been analyzed and then the process of maintaining the elaboration of the system to produce the data transferring is proposed. As per results of robustness, data reduction analysis, fault analysis, time of execution, delay, and the jitter comparison has been analyzed based on the information sharing. This article helps in analyzing the data and the delivery of the information without the traffic in the network is done. Here the traffic can be controlled by creating the network and the routers for sending information. In future based on the packets many n numbers of packets are also can be transfer without the traffic in the network can be proposed later on. The proposed system elaborates the Algorithm for improving fault isolation, Algorithm for improving fault detection, Algorithm for improving data redundancy, Algorithm for improving robustness against the Coalescing novel QoS Routing and fault tolerance improving QoS parameters in the ad-hoc wireless network using CRAFT protocol. The proposed system has fault isolation 7 levels is better than the other existing system. Fault detection is made 3 times better than the early existing system. The Execution time is 96% better, with a reduced delay is approximately less than ± 2 ms. The data redundancy rate is probably nearly 90% confined to the existing services like robustness becoming 93% better, Jitter becoming 7% better, and packet arrival time almost took the ± 50 ms for each transmission.

RETRACTED ARTICLE

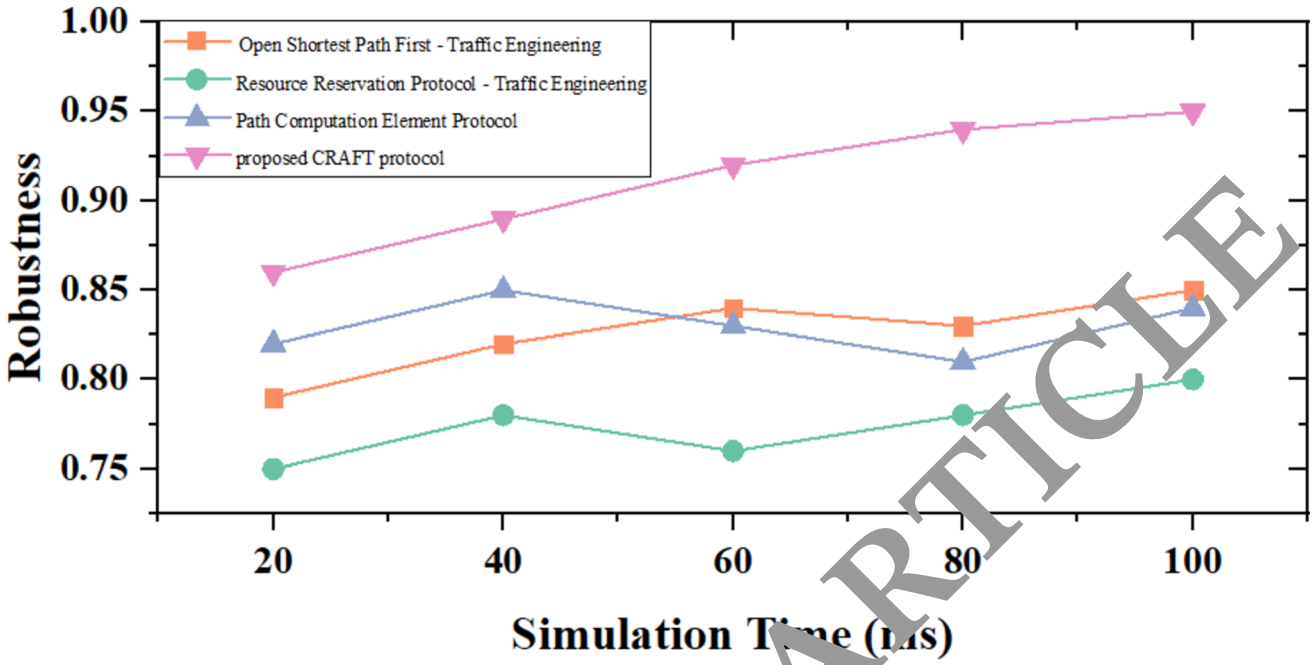



**Fig. 14** Performance Comparison of Network Routing Protocols using robustness factor

RETRACTED ARTICLE

**Author contributions** All author is contributed to the design and methodology of this study, the assessment of the outcomes and the writing of the manuscript.

**Funding** Authors did not receive any funding.

**Data availability** No datasets were generated or analyzed during the current study.

**Code availability** Not applicable.

## Declarations

**Conflict of interest** Authors to no have any conflicts.

## References

1. Bhardwaj, D., Kant, K., & Chauhan, D. S. (2016). Distortion aware node routing protocol for video transmission over MANETs. In *2016 International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET), Chennai, India*, pp. 579–584. https://doi.org/10.1109/WiSPNET.2016.7566201.
2. Barends, R., Kelly, J., Megrant, A., et al. (2014). Superconducting quantum circuits at the surface code threshold for fault tolerance. *Nature, 508*, 500–503. https://doi.org/10.1038/nature13171
3. Sharma, S., & Pathak, D. (2015). Energy aware path formation with link stability in wireless ne network. In *2015 International Conference on Advances in Computer Engineering and Applications, Ghaziabad, India*, pp. 826–831. https://doi.org/10.1109/ICACEA.2015.7164818.
4. Hamerly, R., Bandyopadhyay, S., & Englund, D. (2022). Asymptotically fault-tolerant programmable photonics. *Nature Communications, 13*, 6831. https://doi.org/10.1038/s41467-022-34508-3
5. Song, Y., Guo, C., Xu, P., et al. (2021). Research on routing and scheduling algorithms for the simultaneous transmission of diverse data streaming services on the industrial internet. *Science and Reports, 11*, 18351. https://doi.org/10.1038/s41598-021-97613-9
6. Zhuo, M., Liu, L., Zhou, S., et al. (2021). Survey on security issues of routing and anomaly detection for space information networks. *Science and Reports, 11*, 22261. https://doi.org/10.1038/s41598-021-01638-z
7. Kumar, R., Arya, K. V., Shekhar, S., Agrawal, R. (2014). An on demand routing protocol AODV with end to end reliability and backward route information. In *2014 9th International Conference on Industrial and Information Systems (ICIIS), Gwalior, India*, pp. 1–6. https://doi.org/10.1109/ICIINFS.2014.7036635.
8. Guo, Y., Hu, G., & Shao, D. (2022). QOGMP: QoS-oriented global multi-path traffic scheduling algorithm in software defined network. *Science and Reports, 12*, 14600. https://doi.org/10.1038/s41598-022-18919-w
9. Monika Singh, S., & Wason, A. (2023). Performance investigations on data protection algorithms in generalized multi protocol label switched optical networks. *Scientific Reports, 13*, 425. https://doi.org/10.1038/s41598-022-26942-0
10. Wang, L., Ma, H., Jiang, Y., et al. (2022). A data plane security model of segmented routing based on SDP trust enhancement architecture. *Science and Reports, 12*, 8762. https://doi.org/10.1038/s41598-022-12858-2
11. Agrawal, M., Jain, V., & Mohan, N. (2020). New approach for improving battery power consumption of wireless mobile adhoc networks nodes using genetic algorithm. In *2020 International Conference on Power Electronics & IoT Applications in Renewable Energy and its Control (PARC), Mathura, India*, pp. 378–381. https://doi.org/10.1109/PARC49193.2020.236630.
12. Marwah, G. P. K., & Jain, A. (2022). A hybrid optimization with ensemble learning to ensure VANET network stability based on performance analysis. *Science and Reports, 12*, 10287. https://doi.org/10.1038/s41598-022-14255-1

13. Song, Y., Luo, W., Xu, P., et al. (2022). An improved Lagrangian relaxation algorithm based SDN framework for industrial internet hybrid service flow scheduling. *Science and Reports, 12*, 3861. https://doi.org/10.1038/s41598-022-07125-3
14. Bijur, G., Ramakrishna, M., & Kotegar, K. A. (2021). Multicast tree construction algorithm for dynamic traffic on software defined networks. *Science and Reports, 11*, 23084. https://doi.org/10.1038/s41598-021-02292-1
15. Rrushi, J. L. (2021). DNIC architectural developments for 0-knowledge detection of OPC malware. In *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 1, pp. 30–44. https://doi.org/10.1109/TDSC.2018.2872536.
16. Manivannan, S., Li, W., Zhang, J., Trucco, E., & McKenna, S. J. (2018). Structure prediction for gland segmentation with hand-crafted and deep convolutional features. *IEEE Transactions on Medical Imaging, 37*(1), 210–221. https://doi.org/10.1109/TMI.2017.2750210
17. Steane, A. (1999). Efficient fault-tolerant quantum computing. *Nature, 399*, 124–126. https://doi.org/10.1038/20127
18. Quan, Y. (2022). Improving bitcoin's post-quantum transaction efficiency with a novel lattice-based aggregate signature scheme based on CRYSTALS-dilithium and a STARK protocol. *IEEE Access, 10*, 132472–132482. https://doi.org/10.1109/ACCESS.2022.3227394
19. Moreau, L., Conchon, E., & Sauveron, D. (2021). CRAFT: A continuous remote attestation framework for IoT. *IEEE Access, 9*, 46430–46447. https://doi.org/10.1109/ACCESS.2021.3067697
20. Bartolucci, S., Birchall, P., Bombín, H., et al. (2023). Fusion-based quantum computation. *Nature Communications, 14*, 912. https://doi.org/10.1038/s41467-023-36493-1
21. Cai, C., Weng, J., Yuan, X., & Wang, C. (2021). Enabling reliable keyword search in encrypted decentralized storage with fairness. In *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 1, pp. 131–144. 2021. https://doi.org/10.1109/TDSC.2018.2877332.
22. Antoniazzi, F., & Viola, F. (2019). Building the semantic web of things through a dynamic ontology. *IEEE Internet of Things Journal, 6*(6), 10560–10579. https://doi.org/10.1109/JIOT.2019.2939882
23. Noiri, A., Takeda, K., Nakajima, T., et al. (2022). Fast universal quantum gate above the fault-tolerance threshold in silicon. *Nature, 601*, 338–342. https://doi.org/10.1038/s41586-021-04182-y
24. Gottesman, D. (2016). Efficient fault tolerance. *Nature, 540*, 44–45. https://doi.org/10.1038/nature20479
25. Trinh, C., et al. (2020). A novel lightweight block cipher-based mutual authentication protocol for constrained environments. *IEEE Access, 8*, 165536–165550. https://doi.org/10.1109/ACCESS.2020.3021701
26. Roy, S., Ghosh, P., Barua, D., et al. (2020). Motifs enable communication efficiency and fault-tolerance in transcriptional networks. *Science and Reports, 10*, 9628. https://doi.org/10.1038/s41598-020-66573-x
27. Preet, Kaur, R., & Singh, D. (2022). Dimension learning based chimp optimizer for energy efficient wireless sensor networks. *Science and Reports, 12*, 14968. https://doi.org/10.1038/s41598-022-18001-5
28. Egan, L., Debroy, D. M., Noel, C., et al. (2021). Fault-tolerant control of an error-corrected qubit. *Nature, 598*, 281–286. https://doi.org/10.1038/s41586-021-03928-y
29. Souza, A., Zhang, J., Ryan, C., et al. (2011). Experimental magic state distillation for fault-tolerant quantum computing. *Nature Communications, 2*, 169. https://doi.org/10.1038/ncomms1166
30. Yang, H., Zhang, J., Ji, Y., et al. (2016). Experimental demonstration of multi-dimensional resources integration for service provisioning in cloud radio over fiber network. *Science and Reports, 6*, 30678. https://doi.org/10.1038/srep30678

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



**R. Aruna** has obtained her Ph. D degree in Wireless Networks in the year 2019 from Anna University, Chennai. Currently she is an Associate professor in the Department of Computer Science and Engineering, Vel Tech Rangarajan Dr. Sangunthala R & D Institute of Science and Technology, Chennai, Tamilnadu. She has published 30 research articles in International/National Journals and 3 patents. She acts as session chair and resource person in various international conference and FDP. Currently, she was guiding 7 research scholars in the same university. Her area of interest includes Wireless Networks, Data Science and Analytics and Artificial Intelligence.



**Virendra Singh Kushwah** is working as a Senior Assistant Professor and Program Chair in the Cloud Computing & Automation, School of Computing Science and Engineering, VIT Bhopal University. Dr. Kushwah earned his Ph.D. in Computer Science and Engineering from Maharishi Markandeshwar Deemed-to-be University, Ambala (NAAC-A + +) and M.Tech. (Information Security) in CSE from ABV-IIITM, Gwalior (National Importance University). He has more than 15 years academic experience including various administration positions. He has more than 35 research publications and two patents. Dr. Kushwah chaired three international conferences and delivered various invited talks. He is a life time member of IACSIT, IAENG etc. He is also reviewers of SCI journals like Multimedia Tools and Applications, IEEE Intelligent Systems, Wireless Personal Communications etc. and also international conferences. Dr. Kushwah conducted many workshops and seminars. His area of expertise is fault tolerant and load balancing on cloud computing, machine learning, information security.

RETRACTED ARTICLE



**S. Phani Praveen** is working as an Associate Professor in department of CSE, PVPSIT, Vijayawada, Andhra Pradesh, India. He received his Ph.D. Degree for his work on "The Optimization of Allocation of Resources in Multi Cloud Environment" from the Bharathiar State University, India in 2020. He obtained his M. Tech in CSE from the NCET, JNTUK, Kakinada, India. He received his B. Tech from SSIET, JNTU Hyderabad, India. He is a reviewer for some reputed journals like IEEE Access, Journal of Super Computing and many more Web of Science journals. He has 40 + Scopus indexed papers, 1 edited book, 2 Textbooks. His research interests include Cloud Computing, Blockchain and Networks. He also received young researcher award at IESRA Meet 2020 from Elsevier SSRN Research organized by IJIEMR.



**Rahul Pradhan** is a researcher in the field of Statistics, language models, and various allied fields. He was a vast experience in teaching and research, he took courses such as databases, statistics, data analysis, and search engines. He had published papers in peer-reviewed journals, and conferences and had a patent to his accolades. He had also consultant to few companies in domains of data analysis, & blockchain.



**Amit Jaykumar Shinchawade** Working as an Assistant Professor, department of Electronics & Computer Engineering, Sharad Institute of Technology College of Engineering, Yadrav-Ichalkaranji, Maharashtra. Having 4 years of Industrial Experience and 9 years of Academic Experience. Published 10 Research papers in reputed Journals and Published 10 Papers in National and International conferences. Published 2 Indian Patents and 1 UK Design Patent. Published 3 Book chapters in International Proceedings. Life time member of professional societies—The Indian Society for Technical Education (ISTE), Society for Technologically Advanced Materials, India (STAMI), The Institute of Electronics & Telecommunication Engineering (IETE) and International Association of Engineers (IAENG) .



**Renas Rajab Asaad** works as an Assistant Professor at Nawroz University, department of computer science. The specialization is Artificial Intelligence, Neural Networks, Cyber Security, and Big Data. He work as a vice president of the scientific research and development centre, managing Qubahan academic journal, and Polaris global research organization inc., and global journal of scholarly research and trends.



**R. Lakshmana Kumar** had experience in 7 years of teaching in the area of Computer science & Engineering and completed Ph.D. research at Anna University, Chennai, India. He received his Masters in Computer Science & Engineering from SNS College of Technology and Bachelors in Information Technology at Sri Ramakrishna Engineering College. In reputed Conferences and Journals, I have published more than 10 papers in international refereed Peer-reviewed journals and 10 papers in conferences for the credential. My current research includes the security-related issue over data in several domains such as cloud computing, Internet of Things, and Mobile Adhoc Networks, Deep Learning.

RETRACTED ARTICLE

## Authors and Affiliations

**R. Aruna[1] · Virendra Singh Kushwah[2] · S. Phani Praveen[3] · Rahul Pradhan[4] · Amit Jaykumar Chinchawade[5] · Renas Rajab Asaad[6] · R. Lakshmana Kumar[7]**

✉ R. Aruna
arunasnandhini@gmail.com; arunar67@outlook.com

Virendra Singh Kushwah
virendra.kushwah@vitbhopal.ac.in;
kushwah.virendra248@gmail.com

S. Phani Praveen
phani.0713@gmail.com

Rahul Pradhan
rahul.pradhan@gla.ac.in

Amit Jaykumar Chinchawade
amitchinchawade@sitcoe.org.in

Renas Rajab Asaad
renas.rekany@nawroz.edu.krd

R. Lakshmana Kumar
Lakshmanakumar93@gmail.com

[1] Department of Computer Science and Engineering, School of Computing, Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and Technology, Chennai, India

[2] School of Computing Science & Engineering, VIT Bhopal University, Bhopal-Indore Highway, Kothrikalan, Sehore, Madhya Pradesh 466114, India

[3] Department of CSE, PVP Siddhartha Institute of Technology, Vijayawada, Andhra Pradesh, India

[4] GLA University, Mathura, India

[5] Department of Electronics and Computer Engineering, Sharad Institute of Technology College of Engineering, Yadrav (Ichalkaranji), Maharashtra 416121, India

[6] Department of Computer Science, Nawroz University, Duhok, Kurdistan Region, Iraq

[7] Department of Computer Science & Engineering, Tagore Institute of Engineering and Technology, Salem, India

RETRACTED ARTICLE